

CHAPTER 24

Digital Signal Processing (DSP)

Digital signal processing (DSP) is pervasive in modern technologies, ranging from multimedia to communication to various special devices such as automotive, robotics, and medical systems. Figure 24-1 shows a common DSP scheme that uses ADC to quantize an external analog input signal. A low pass analog filter is often used before ADC sampling to remove noises above the Nyquist frequency (one-half of the ADC sampling rate). In the same way, another low pass analog filter is often used after the DAC output to limit output frequencies.

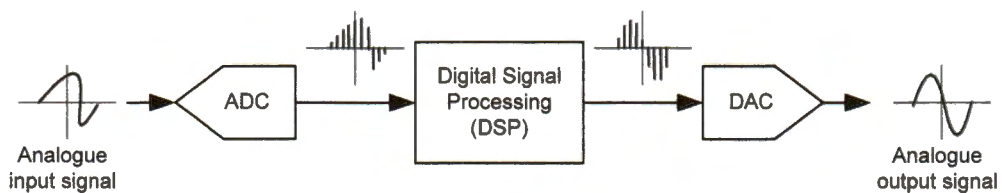


Figure 24-1. Common diagram of a DSP system

24.1 Fixed-point and Floating-point DSP

Based on the way numerical values are represented and operated, DSP systems are divided into two categories: *fixed-point* and *floating-point*. They strike different tradeoffs between accuracy, ease of use, cost, and power consumption.

- Floating-point DSP can express a very wide range of numbers more accurately than fixed-point DSP. For example, high-end audio applications often use floating-point DSP to achieve a high degree of accuracy because human ears are very sensitive.

- Floating-point DSP is easier to program and has a shorter time to market. Due to the nature of floating-point representation, numbers are automatically scaled. Fixed-point DSP often must use assembly instructions or compiler built-in functions. Also, it should take consideration of the data range to avoid overflow and underflow.
- To meet the speed requirement, floating-point DSP often uses high-performance FPU, which increases hardware cost and consumes more power. Fixed-point DSP only uses integer operations, which satisfy the cost and power constraints of many embedded systems.

Therefore, floating-point DSP is suitable for low-volume products, or applications in which software development cost is a serious concern or data range is very dynamic. Example applications are development prototyping, audio applications, medical devices, and military applications. On the contrary, fixed-point DSP is preferred in high-volume products, or applications in which cost and power are of great concern or data range is predictable. Example applications include video applications, speech recognition, and wireless communications.

Cortex-M4 and M7 provide SIMD (*Single-Instruction Multiple-Data*) instructions to accelerate fixed-point DSP performance cost-effectively. This chapter introduces these instructions in detail.

24.2 Fixed-point Data Types in DSP

As discussed in Chapter 12, signed fixed-point numbers are denoted as Q_n , where n is the number of fractional bits. A Q_n number is a signed $(n + 1)$ -bit two's complement integer, with the fixed-point value represented as that integer divided by 2^n .

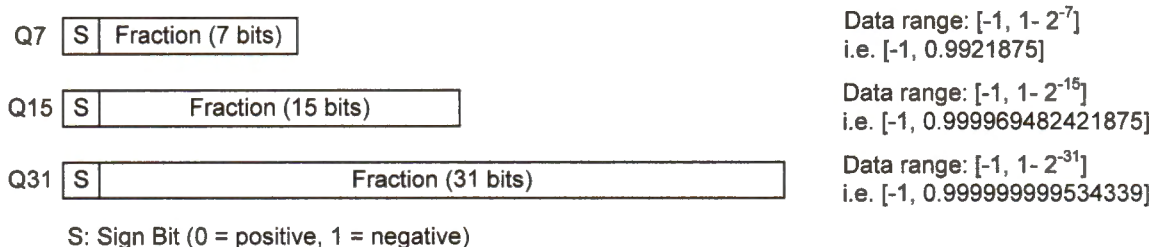


Figure 24-2. Signed fixed-point format Q7, Q15, and Q31

Fixed-point formats commonly used in DSP include Q7, Q15, and Q31, as shown in Figure 24-2. A value in Q7, Q15, and Q31 can be stored in a byte, halfword, and word,

respectively. They can represent numbers between -1 and 1 (excluding 1). These formats align memory byte boundary, and thus it is efficient to load numbers from or store them in the data memory.

Software treats a signed fixed-point number as if it were a signed integer represented in two's complement. In C, we declare the data type of signed fixed-point variables as signed integers, as shown below.

```
// 8-bit fractional data type in Q7 format
int8_t arrayA[1024]; // Q7 array

// 16-bit fractional data type in Q15 format
int16_t arrayB[1024]; // Q15 array

// 32-bit fractional data type in Q31 format
int32_t arrayC[1024]; // Q31 array
```

Example 24-1. Declaring signed fixed-point variables as signed integers in C

We can use shift operations to convert a value between Q7, Q15, and Q31 format, as shown in Figure 24-3. Because they are signed, ASR should be used to shift right.

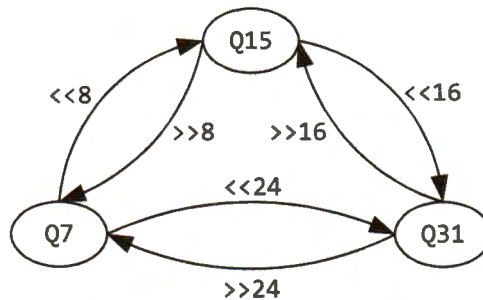


Figure 24-3. Conversion between Q7, Q15, and Q31

24.3 Saturation

Processors treat fixed-point numbers as integers with a fixed range.

- When a result of an arithmetic operation is larger than the maximum representable number, *overflow* occurs.
- When a result is smaller than the minimum representable number, *underflow* occurs. In ARM technical documents, underflow is often called as overflow.

When two's complement is used to represent signed fixed-point numbers, overflow and underflow result in a wrap-around phenomenon (see Figure 24-5). This generates

significant errors, and the results can be catastrophic in DSP applications. Therefore, DSP processors often support arithmetic operations with saturation.

- Without saturation, overflow and underflow values are truncated, and only the least significant bits of the result are retained. Thus, overflow values are trimmed to very small values, and underflow values are wrapped around to very high values, leading to a wrap-around phenomenon (see Figure 24-5).
- When overflow or underflow occurs, saturation operation clamps the result to the maximum or minimum limit of the representable range (see Figure 24-6).

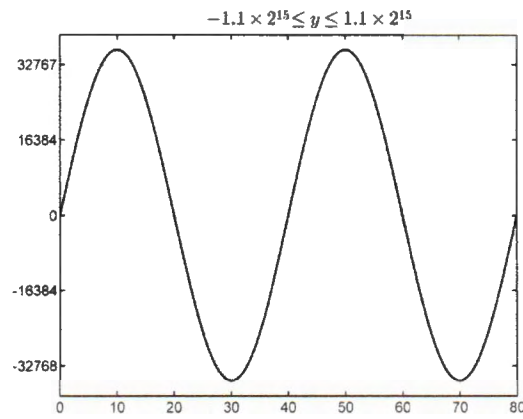


Figure 24-4. Equivalent integer values of a signal. Overflow and underflow occur if represented in Q15

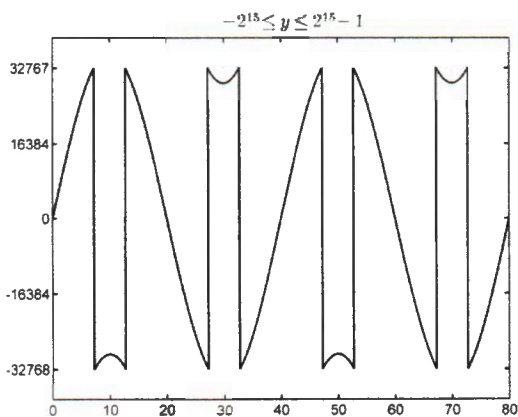


Figure 24-5. Wrapped-around output if there is no saturation

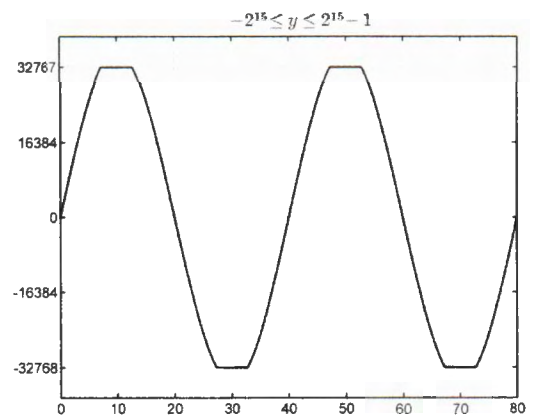


Figure 24-6. Saturated output

Chapter 4.4 introduces SSAT and USAT that saturate a 32-bit integer to a specific signed or unsigned range. The following are two instructions that saturate two halfwords of a register in parallel.

SSAT16 Rd,#imm4,Rm	Signed saturate two signed 16-bit values #imm4 =saturation bit position, $-2^{\text{imm4}-1} \leq x \leq 2^{\text{imm4}-1}-1$
USAT16 Rd,#imm4,Rm	Unsigned saturate two unsigned 16-bit values #imm4 = saturation bit position, $0 \leq x \leq 2^{\text{imm4}} - 1$

Table 24-1. Saturating two 16-bit values in parallel

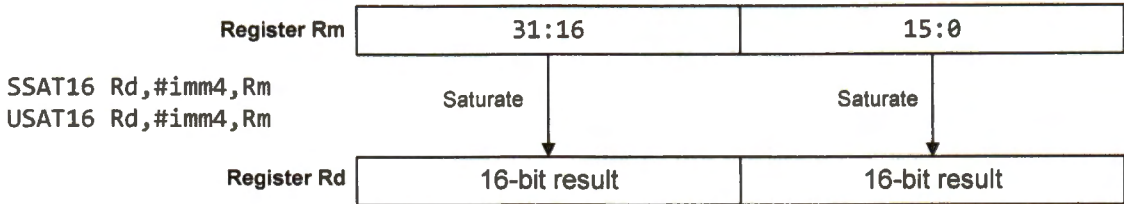


Figure 24-7. Signed and unsigned saturation of two 16-bit operands

When overflow or underflow occurs, the Q bit flag in the application program status register (APSR) is set. Software can use the MRS instruction to read the Q flag and the MSR instruction to clear the Q flag. The Q flag is sticky in that once it has been set, it remains set until software explicitly clears it.

Read Q flag	Clear Q flag
MRS r0, APSR TST r0, #(1<<27) ; Z is 1 if Q is 0	MRS r0, APSR ; Read APSR BIC r0, r0, #(1<<27) ; Clear Q flag MSR APSR_nzcvq, r0 ; Write APSR

Example 24-2. Assembly code to read or clear Q flag.

24.4 Arithmetic Instructions

For DSP applications, Table 24-2 summarizes arithmetic instructions available in Cortex-M4 and M7. Note that only ADD, ADC, SUB, MUL, SDIV, UDIV, UMULL, SMULL, MLA, MLS, SMLAL, and UMULAL are available in Cortex-M.

Many of the instructions listed in Table 24-2 are SIMD (Single-Instruction Multiple-Data). SIMD instructions leverage data-level parallelism and simultaneously apply the same operation to each member of a set of data.

There exists much data-level parallelism in many applications such as multimedia, graphics, and encryption. For example, to increase the brightness, a constant value is added to the red, green, and blue values of each pixel in an image. This SIMD-based approach speeds up the computation cost-effectively.

Data Length & Operation	Instructions
$8 \pm 8 = 8$	SADD8, QADD8, USADD8, UQADD8, SSUB8, QSUB8, USUB8, UQSUB8
$(8 \pm 8) \div \text{two} = 8$	SHADD8, UHADD8, SHSUB8, UHSUB8
$16 \pm 16 = 16$	SADD16, QADD16, USADD16, UQADD16, SSUB16, QSUB16, USUB16, UQSUB16, SASX, QASX, UASX, UQASX, SSAX, QSAX, USAX, UQSAX
$(16 \pm 16) \div \text{two} = 16$	SHADD16, UHADD16, SHSUB16, UHSUB16, SHASX, UHASX, SHSAX, UHSAX
$32 \pm 32 = 32$	QADD, QSUB, ADD, ADC, SUB, ADC
$32 \pm 32 \times \text{two} = 32$	QDADD, QDSUB
$16 \times 16 = 32$	SMULBB, SMULBT, SMULTB, SMULTT
$16 \times 32 = 32$	SMULWB, SMULWT,
$32 \times 32 = 32$	SMMUL, SMMULR, MUL
$32 \times 32 = 64$	UMULL, SMULL
$32 \div 32 = 32$	SDIV, UDIV
$32 + 16 \times 16 = 32$	SMLABB, SMLABT, SMLATB, SMLATT
$32 \pm 16 \times 32 = 32$	SMLAWB, SMLAWT
$32 \pm 32 \times 32 = 32$	SMMLA, SMMLAR, SMMLS, SMMLSR, MLA, MLS
$64 + 16 \times 16 = 64$	SMLALBB, SMLALBT, SMLALTB, SMLALTT
$64 + 32 \times 32 = 64$	SMLAL, UMULAL
$16 \times 16 \pm 16 \times 16 = 32$	SMUAD, SMUADX, SMUSD, SMUSDX
$32 + 16 \times 16 \pm 16 \times 16 = 32$	SMLAD, SMLADX, SMLSD, SMLSDX
$64 + 16 \times 16 \pm 16 \times 16 = 64$	SMLALD, SMLALDX, SMLSLD, SMLSLDX
$16 \times 16 + 32 + 32 = 64$	UMAAL
$\sum_{i=1}^4 \text{absolute}(8 - 8)_i$	USAD8
$32 + \sum_{i=1}^4 \text{absolute}(8 - 8)_i$	USADA8

Table 24-2. Arithmetic instruction with different data sizes

Figure 24-8 shows an example SIMD instruction (SADD8) that adds four pairs of 8-bit signed integers in parallel. This approach has several advantages.

- First, parallel arithmetic operations accelerate the computation speed.
- Second, these concurrent operations amortize the overhead of instruction fetching, instruction decoding and data memory accessing.
- Last but not the least, a less number of branch instructions are required in a loop. Branch instructions are detrimental to performance. Modern microprocessors execute instructions in a pipeline fashion. When executing a branch instruction, a processor has to stall the execution pipeline if there is no branch prediction or flush the execution pipeline when a branch prediction turns out to be incorrect.

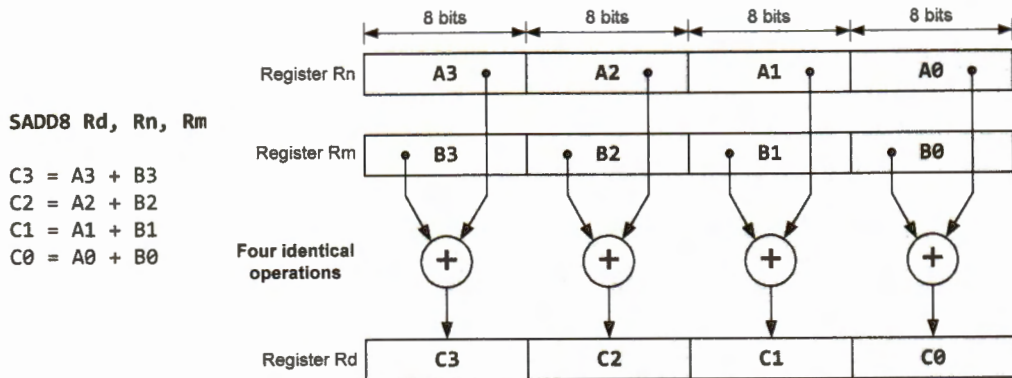


Figure 24-8. Example SIMD instruction (SADD8) that adds four pairs of 8-bit signed integers in parallel. The results are packed into one destination register (Rd).

Table 24-2 summarizes all SIMD addition and subtraction instructions. The following sections give a detailed explanation of these SIMD instructions.

Prefix/ Suffix	Signed (S)	Signed Saturating (Q)	Signed Halving (SH)	Unsigned (U)	Unsigned Saturating (UQ)	Unsigned Halving (UH)
ADD8	SADD8	QADD8	SHADD8	UADD8	UQADD8	UHADD8
SUB8	SSUB8	QSUB8	SHSUB8	USUB8	UQSUB8	UHSUB8
ADD16	SADD16	QADD16	SHADD16	UADD16	UQADD16	UHADD16
SUB16	SSUB16	QSUB16	SHSUB16	USUB16	UQSUB16	UHSUB16
ASX	SASX	QASX	SHASX	UASX	UQASX	UHASX
SAX	SSAX	QSAX	SHSAX	USAX	UQSAX	UHSAX

Table 24-3. Summary of 8- and 16-bit SIMD instructions for addition and subtraction

24.4.1 Parallel 8-bit Add and Subtract

As discussed previously, computers treat fixed-point numbers as if they were integers. The following lists instructions that add or subtract four pairs of 8-bit integers.

SADD8	Rd,Rn,Rm	Signed add 4 pairs of 8-bit integers
SSUB8	Rd,Rn,Rm	Signed subtract 4 pairs of 8-bit integers
UADD8	Rd,Rn,Rm	Unsigned add 4 pairs of 8-bit integers
USUB8	Rd,Rn,Rm	Unsigned subtract 4 pairs of 8-bit integers
QADD8	Rd,Rn,Rm	Saturating add 4 pairs of signed 8-bit integers
QSUB8	Rd,Rn,Rm	Signed saturating subtract 4 pairs of 8-bit integers
UQADD8	Rd,Rn,Rm	Unsigned saturating add 4 pairs of 8-bit integers
UQSUB8	Rd,Rn,Rm	Unsigned saturating subtract 4 pairs of 8-bit integers

Table 24-4. Parallel 8-bit add and subtract

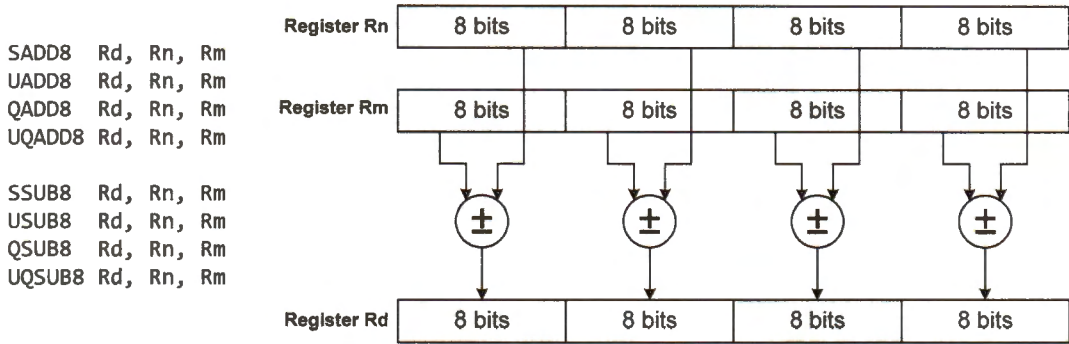


Figure 24-9. Add or subtract 8-bit integers

Example 24-3 shows a simple C program, which adds the corresponding Q7 signed fixed-point numbers in two arrays. It executes the loops 1024 times.

```

int8_t A[1024], B[1024], C[1024]; // Q7 signed fixed-point
int i;

for (i = 0; i < 1024; i++){
    C[i] = A[i] + B[i];
}
    
```

Example 24-3. A C program that does not use SIMD

We can use SADD8 to speed up pairwise addition in the above C program. Example 24-4 shows the assembly implementation. This assembly program makes only 256 iterations, instead of 1024. It completes four add operations in each iteration.

```

LDR    r3, =A           ; Address of array A
LDR    r4, =B           ; Address of array B
LDR    r5, =C           ; Address of array C

MOV    r6, #256         ; Loop counter

loop LDR    r0, [r3], #4   ; Load four Q7 numbers, post-index: r3 = r3 + 4
      LDR    r1, [r4], #4   ; Load four Q7 numbers, post-index: r4 = r4 + 4
      SADD8 r2, r0, r1      ; Add four pairs of Q7 numbers
      STR    r2, [r5], #4   ; Store four Q7 numbers, post-index: r5 = r5 + 4
      SUBS   r6, r6, #1     ; Decrement loop counter
      BGT    loop
    
```

Example 24-4. An assembly program that uses SIMD

If the array size is not exactly divisible by four, a compensation loop is needed to handle the addition of remainder Q7 values after the main loop shown in Example 24-4. In the compensation loop, we can load each byte and add them up. To simplify the description

of example codes, we assume the size of Q7 arrays is always a multiple of four and the size of Q15 arrays is always a multiple of two.

A C program can also call the compiler's built-in function (also called intrinsic function) `__SADD8`, as shown in Example 24-5, to utilize the SIMD capability of a microprocessor.

```
int8_t A[1024], B[1024], C[1024];

uint32_t *pA, *pB, *pC, i;

pA = (uint32_t*) A;    // Pointer-to-pointer casting. It changes compiler's
pB = (uint32_t*) B;    // views on array A, B and C from signed 8-bit
pC = (uint32_t*) C;    // integers to unsigned 32-bit integers

for(i = 0; i < 256; i++) {
    *pC = __SADD8(*pA, *pB); // Call compiler built-in function which
                             // calls the assembly SADD8 instruction
    pA++;                    // Pointer is incremented by size of uint32_t (i.e., 4),
    pB++;                    // because compiler views them as arrays of 32-bit
    pC++;                    // unsigned integers
}
```

Example 24-5. A C program that uses SIMD

Example 24-6 gives the implementation of the `__SADD8` function.

```
uint32_t __SADD8(uint32_t op1, uint32_t op2){

    uint32_t result;

    __ASM {
        sadd8 result, op1, op2
    }

    return result;
}
```

Example 24-6. Implementation of the `__SADD8` function

The results of parallel 8-bit add and subtract can be halved by using the following instructions, as shown in Figure 24-10.

SHADD8 Rd,Rn,Rm	Signed halving add 4 pairs of 8-bit integers
SHSUB8 Rd,Rn,Rm	Signed halving subtract 4 pairs of 8-bit integers
UHADD8 Rd,Rn,Rm	Unsigned halving add 4 pairs of 8-bit integers
UHSUB8 Rd,Rn,Rm	Unsigned halving subtract 4 pairs of 8-bit integers

Table 24-5. Parallel 8-bit instructions to add (or subtract) and division by two

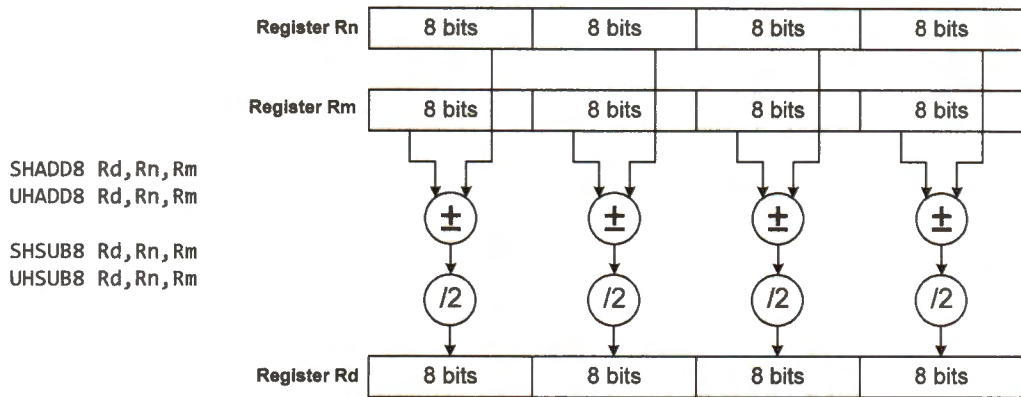


Figure 24-10. Add or subtract 8-bit values, and divide results in half

No 8-bit saturating addition and subtraction instructions change the Q flag in APSR when overflow occurs. Note that in ARM technical documents, underflow is distinguished from overflow, and underflow is referred to as overflow.

```
LDR    r0, =0x80808080
UQADD8 r2, r0, r0      ; r2 = 0xFFFFFFFF
                        ; unsigned saturating, 0x80 + 0x80 = 0xFF
                        ; Even though overflow occurs, Q flag is not changed

LDR    r0, =0x70707070
QADD8  r2, r0, r0      ; r2 = 0x7F7F7F7F,
                        ; signed saturating, 0x70 + 0x70 = 0x7F
                        ; Even though overflow occurs, Q flag is not changed

LDR    r0, =0x81818181
QADD8  r2, r0, r0      ; r2 = 0x80808080,
                        ; signed saturating, 0x81 + 0x81 = 0x80
                        ; Even though overflow occurs, Q flag is not changed

LDR    r1, =0x80808080
LDR    r2, =0x70707070
QSUB8  r3, r1, r2      ; r3 = 0x80808080,
                        ; signed saturating, 0x80 - 0x70 = 0x80
                        ; Even though overflow occurs, Q flag is not changed

QSUB8  r3, r2, r1      ; r3 = 0x7F7F7F7F,
                        ; signed saturating, 0x70 - 0x80 = 0x7F
                        ; Even though overflow occurs, Q flag is not changed

UQSUB8 r3, r2, r1      ; r3 = 0x00000000,
                        ; unsigned saturating, 0x70 - 0x80 = 0x00
                        ; Even though overflow occurs, Q flag is not changed
```

Example 24-7. Updating Q flags

24.4.2 Parallel 16-bit Add and Subtract

A 16-bit parallel add or subtract instruction carries out pairwise operations on halfwords, as shown in Figure 24-11.

SADD16 Rd, Rn, Rm	Signed add two pairs of 16-bit integers $Rd[T] = \text{truncate16}(Rn[T] + Rm[T])$ $Rd[B] = \text{truncate16}(Rn[B] + Rm[B])$
SSUB16 Rd, Rn, Rm	Signed subtract two pairs of 16-bit integers $Rd[T] = \text{truncate16}(Rn[T] - Rm[T])$ $Rd[B] = \text{truncate16}(Rn[B] - Rm[B])$
UADD16 Rd, Rn, Rm	Unsigned add two pairs of 16-bit integers $Rd[T] = \text{truncate16}(Rn[T] + Rm[T])$ $Rd[B] = \text{truncate16}(Rn[B] + Rm[B])$
USUB16 Rd, Rn, Rm	Unsigned subtract two pairs of 16-bit integers $Rd[T] = \text{truncate16}(Rn[T] - Rm[T])$ $Rd[B] = \text{truncate16}(Rn[B] - Rm[B])$
QADD16 Rd, Rn, Rm	Saturating add two pairs of signed 16-bit integers $Rd[T] = \text{signed_saturate16}(Rn[T] + Rm[T])$ $Rd[B] = \text{signed_saturate16}(Rn[B] + Rm[B])$
QSUB16 Rd, Rn, Rm	Signed saturating subtract 2 pairs of 16-bit integers $Rd[T] = \text{signed_saturate16}(Rn[T] - Rm[T])$ $Rd[B] = \text{signed_saturate16}(Rn[B] - Rm[B])$
UQADD16 Rd, Rn, Rm	Unsigned saturating add 2 pairs of 16-bit integers $Rd[T] = \text{unsigned_saturate16}(Rn[T] + Rm[T])$ $Rd[B] = \text{unsigned_saturate16}(Rn[B] + Rm[B])$
UQSUB16 Rd, Rn, Rm	Unsigned saturating subtract 2 pairs of 16-bit integers $Rd[T] = \text{unsigned_saturate16}(Rn[T] - Rm[T])$ $Rd[B] = \text{unsigned_saturate16}(Rn[B] - Rm[B])$

Table 24-6. Parallel 16-bit add and subtract

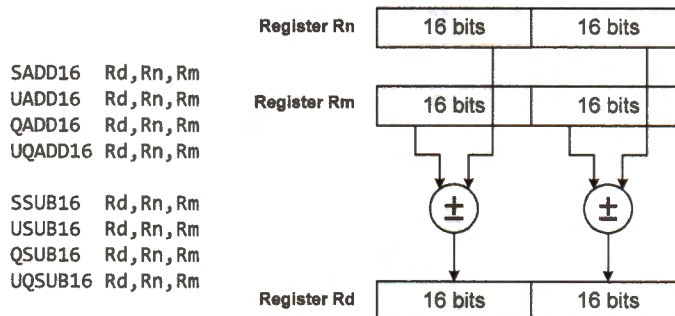


Figure 24-11. Add or subtract 16-bit values

The results can also be halved, as shown in Figure 24-12. Because saturation never occurs on add or subtract with dividing in half, there are no corresponding halving instructions for saturating add or subtract.

SHADD16 Rd,Rn,Rm	Signed halving add two pairs of 16-bit integers $Rd[T] = (Rn[T] + Rm[T])/2$ $Rd[B] = (Rn[B] + Rm[B])/2$
SHSUB16 Rd,Rn,Rm	Signed halving subtract two pairs of 16-bit integers $Rd[T] = (Rn[T] - Rm[T])/2$ $Rd[B] = (Rn[B] - Rm[B])/2$
UHADD16 Rd,Rn,Rm	Unsigned halving add two pairs of 16-bit integers $Rd[T] = (Rn[T] + Rm[T])/2$ $Rd[B] = (Rn[B] + Rm[B])/2$
UHSUB16 Rd,Rn,Rm	Unsigned halving subtract two pairs of 16-bit integers $Rd[T] = (Rn[T] - Rm[T])/2$ $Rd[B] = (Rn[B] - Rm[B])/2$

Table 24-7. Parallel 16-bit add (or subtract), and division by two

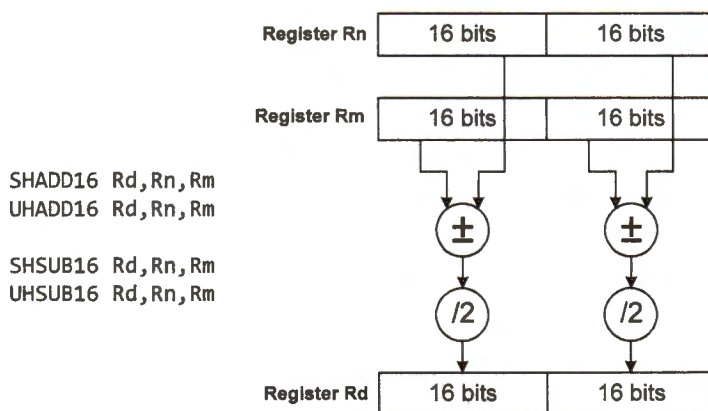


Figure 24-12. Add or subtract two pairs of halfwords, and divide results in half

All 16-bit saturating subtraction and addition instructions cannot set the Q flag in APSR.

```

LDR    r0,=0x80008000
UQADD16 r2,r0,r0          ; r2=0xFFFFFFFF,
                          ; unsigned saturating, 0x8000+0x8000=0xFFFF

LDR    r0,=0x70007000
QADD16 r2,r0,r0           ; r2=0xFFFFFFFF,
                          ; signed saturating, 0x7000+0x7000=0x7FFF

LDR    r1, =0x80008000
LDR    r2, =0x70007000

QSUB16 r3, r1, r2         ; r2=0x80008000,
                          ; signed saturating, 0x8000-0x7000=0x8000

QSUB16 r3, r2, r1         ; r2=0x7FFF7FFF,
                          ; signed saturating, 0x7000-0x8000=0x7FFF

```

24.4.3 32-bit Add and Subtract

Saturating add and subtract instructions (QADD and QSUB) for 32-bit integers is provided to eliminate the wrap-around phenomena. Note that they are not SIMD instructions because each of them processes only one pair of data elements. The Q flag is set if saturation occurs.

QADD Rd,Rn,Rm	Saturating add signed 32-bit integers Rd = saturate32(Rn + Rm)
QSUB Rd,Rn,Rm	Signed saturating subtract two 32-bit integers Rd = saturate32(Rn - Rm)

In the following example, the QADD and QSUB operations set the Q flag. Once the Q flag is set, it remains as 1 until software explicitly clears it.

```

LDR  r0, =0x70000000
ADD  r3, r0, r0      ; r3 = 0xE0000000, Q is unchanged
QADD r4, r0, r0      ; r4 = 0x7FFFFFFF, Q = 1

; Q flag is sticky, and thus software has to clear it explicitly
MRS  r0, APSR        ; Read flags
BIC  r0, r0, #(1<<27) ; Clear Q flag
MSR  APSR_nzcvq, r0  ; Store flags

LDR  r1, =0x80000000
LDR  r2, =0x70000000
SUB  r3, r1, r2      ; r3 = 0x10000000
QSUB r4, r1, r2      ; r4 = 0x80000000, Q = 1

```

With saturation, the second operand can also be doubled before it is added to or subtracted from the first operand. The saturation is applied to both doubling and addition/subtraction. The Q flag is set if saturation occurs on doubling or addition/subtraction.

QDADD Rd,Rn,Rm	Saturating double and add Rd = saturate32(Rn + saturate32(Rm * 2))
QDSUB Rd,Rn,Rm	Saturating double and subtract Rd = saturate32(Rn - saturate32(Rm * 2))

The double and add instruction (QDADD) is often used in a binary method to multiply a constant and a huge number such as a 256-bit number. Given a very large n -bit integer $K = \sum_{i=0}^n (k_i 2^i)$, where $k_i \in \{0, 1\}$, the product $K \times C$ can be written as

$$P = K \times C = \sum_{i=0}^{n-1} k_i \times (2 \times C)$$

The basic idea is that the program scans all bits of K starting from the most significant bit. Each individual bit of K determines the computation. If $k_i = 0$, the product is doubled; otherwise, a double and add operation is performed on the product if $k_i = 1$. Such multiplication method is called the binary double-and-add algorithm. Example 24-8 gives the pseudocode of its implementation.

```

P = 0; // Initialization

for i from n-1 down to 0

    if(k(i) == 1) then
        P = 2*P + C
    else
        P = 2*P
    endif

endfor

```

Example 24-8. Pseudocode of double-and-add algorithm to multiply K and C

24.4.4 Sum of Absolute Difference

There are two special instructions, USAD8, and USADA8, which calculate the unsigned sum (or accumulated sum) of the absolute difference (SAD) between four pairs of 8-bit integers in parallel.

USAD8	Rd,Rn,Rm	Unsigned sum of absolute differences
USADA8	Rd,Rn,Rm,Ra	Unsigned sum of absolute differences and accumulate

The following equation defines the sum of absolute difference (SAD):

$$SAD = \sum_{i=1}^4 absolute(a_i - b_i)$$

Computing the sum of absolute difference is often needed in block-based motion estimation in video compression. A video frame is divided into blocks of equal size. Motion estimation needs to calculate the similarity of blocks in two adjacent frames. The sum of absolute difference between a block in the current frame and a block in the last frame is often used to measure the similarity between these two blocks. In a video frame, each pixel typically has 24 bits, with 8 bits for each of red, green, and blue values.

Two instructions USAD8 and USADA8 can be used to calculate the absolute differences between four pairs of 8-bit unsigned values in parallel. USADA8 adds the sum of the absolute difference in a 32-bit unsigned accumulator.

By leveraging the data-level parallelism, we can use USADA8 to calculate the sum of absolute difference efficiently, as shown in Figure 24-8.

Compute the sum of absolute difference	C Function with inline assembly
<pre>// Two 16-by-16 data blocks uint8_t blk1[16][16]; uint8_t blk2[16][16]; uint32_t *a, *b; // accessing 4 bytes uint32_t sad = 0; for(i = 0; i < 16; i++){ for(j = 0; j < 4; j++){ a = & blk1[i][4*j]; b = & blk2[i][4*j]; sad += __USADA8(*a, *b, sad); } }</pre>	<pre>uint32_t __USADA8(uint32_t op1, uint32_t op2, uint32_t sad) { __ASM { USADA8 sad, op1, op2, sad } return sad; }</pre>

Example 24-9. Calculating the sum of absolute difference between two 16-by-16 blocks

24.4.5 Extension and Add

As introduced in Chapter 4.8, instructions SXTB, SXTB, UXTB, and UXTH perform signed or unsigned extension, which increases the bit width from a byte or halfword to a word.

- Signed extension fills in additional upper bits with 0 if the sign bit is 0, and fills those upper bits with 1 if the sign bit is 1, to preserve the sign and value.
- Unsigned extension, also called zero extension, pads the source operand with zeros from the left side to 32 bits.

When adding two numbers with different bit lengths, the extension and addition operations can be performed in one instruction, as shown below. Note that only ROR is allowed for the second source operand. The other shift operations, including LSL, LSR, ASR, and RRX, are not permitted in these instructions.

SXTAB Rd,Rn,Rm{,ROR #}	Rotate, signed-extend a byte to word, and add $Rd = Rn + \text{sign_extend}((Rm, \text{ROR } \#)[7:0])$
UXTAB Rd,Rn,Rm{,ROR #}	Rotate, unsigned-extend a byte to word, and add $Rd = Rn + \text{zero_extend}((Rm, \text{ROR } \#)[7:0])$
SXTAH Rd,Rn,Rm{,ROR #}	Rotate, signed-extend a halfword to word, and add $Rd = Rn + \text{sign_extend}((Rm, \text{ROR } \#)[15:0])$
UXTAH Rd,Rn,Rm{,ROR #}	Rotate, unsigned extend halfword to word, and add $Rd = Rn + \text{zero_extend}((Rm, \text{ROR } \#)[15:0])$

For example, we can add two 16-bit signed integers as follows:

```
LDR    r5, [r0]           ; Load two 16-bit values
SXTAH  r3, r3, r5          ; sum += sign_extend(r5[B]), r3 = sum
SXTAH  r3, r3, r5, ROR #16 ; sum += sign_extend(r5[T]), r3 = sum
```

For unsigned 16-bit integers, we should use UXTAH with zero extension:

```
LDR    r5, [r0]           ; Load two 16-bit values
UXTAH  r3, r3, r5          ; sum += zero_extend(r5[B]), r3 = sum
UXTAH  r3, r3, r5, ROR #16 ; sum += zero_extend(r5[T]), r3 = sum
```

Similarly, we can use UXTAB to add four 8-bit unsigned integers:

```
LDR    r5, [r0]           ; Load four 8-bit values
UXTAB  r3, r3, r5          ; sum += zero_extend(r5[7:0]), r3 = sum
UXTAB  r3, r3, r5, ROR #8  ; sum += zero_extend(r5[15:8]), r3 = sum
UXTAB  r3, r3, r5, ROR #16 ; sum += zero_extend(r5[23:16]), r3 = sum
UXTAB  r3, r3, r5, ROR #24 ; sum += zero_extend(r5[31:24]), r3 = sum
```

As shown in Figure 24-13, Cortex-M4 and M7 provide special SIMD instructions that first extract two bytes from the second operand, then extend them to halfwords, and finally add them to the corresponding halfwords in the first operand.

SXTAB16 Rd,Rn,Rm{,ROR #}	Rotate, dual extend 8 bits to 16 bits, and add $Rd[T] = Rn[T] + \text{sign_extend}((Rm, \text{ROR \#})[23:16])$ $Rd[B] = Rn[B] + \text{sign_extend}((Rm, \text{ROR \#})[7:0])$
UXTAB16 Rd,Rn,Rm{,ROR #}	Rotate, dual extend 8 bits to 16 bits, and add $Rd[T] = Rn[T] + \text{zero_extend}((Rn, \text{ROR \#})[23:16])$ $Rd[B] = Rn[B] + \text{zero_extend}((Rn, \text{ROR \#})[7:0])$

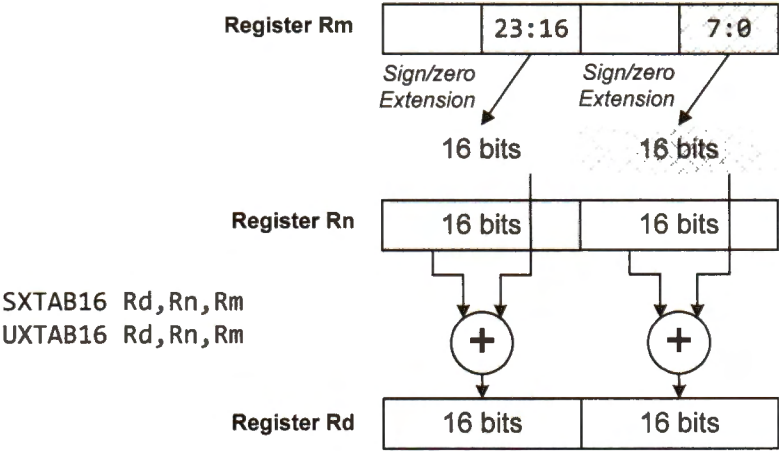


Figure 24-13. Dual extension from bytes to halfwords, and dual addition

In the following program, we can add the lower two 8-bit signed integers. At the same time, the upper two 8-bit signed integers are also added up. The sums are saved in the upper and lower halfword of the destination register.

```
LDR    r5, [r0]           ; Load four 8-bit values
SXTAB16 r3, r3, r5        ; sum[B] += sum[B] + sign_extend(r5[7:0])
                        ; sum[T] += sum[T] + sign_extend(r5[23:16])
SXTAB16 r3, r3, r5, ROR #8 ; sum[B] += sum[B] + sign_extend(r5[15:8])
                        ; sum[T] += sum[T] + sign_extend(r5[31:24])
```

24.4.6 Add and Subtract Halfwords with Exchange

In many DSP algorithms, such as Fast Fourier Transform (FFT), the sum and the difference of numbers are needed for subsequent processing. In the traditional approach, an addition and subtraction are executed in a serial fashion, which limits the throughput. To accelerate the computation, Cortex-M4 and M7 provide SIMD instructions that can perform one addition and one subtraction simultaneously, as listed in Table 24-8, Table 24-9, and Table 24-10.

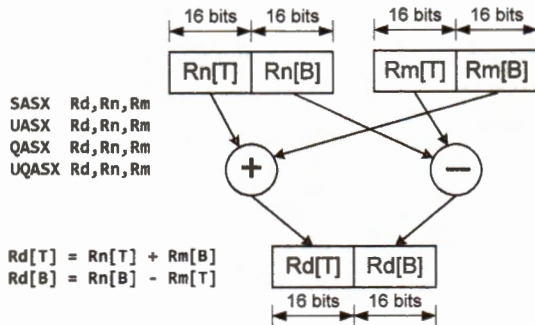
- The prefix S, U, and Q stand for signed, unsigned, and saturating, respectively.
- In the instruction, A stands for add, S for subtracting, H for halving, and X for exchange.
- For unsigned saturation instructions, including UQASX and UQSAX, the results are saturated in the range $[-2^{15}, 2^{15}-1]$. For signed saturation instructions, including QASX and QSAX, the results are saturated in the range $[0, 2^{16}-1]$. However, these instructions do not change the Q flag even if saturation has occurred.

SASX Rd, Rn, Rm	Signed add and subtract with exchange Rd[T] = truncate16(Rn[T] + Rm[B]) Rd[B] = truncate16(Rn[B] - Rm[T])
UASX Rd, Rn, Rm	Unsigned add and subtract with exchange Rd[T] = truncate16(Rn[T] + Rm[B]) Rd[B] = truncate16(Rn[B] - Rm[T])
QASX Rd, Rn, Rm	Saturating add and subtract with exchange Rd[T] = saturate16(Rn[T] + Rm[B]) Rd[B] = saturate16(Rn[B] - Rm[T])
UQASX Rd, Rn, Rm	Unsigned saturating add and subtract with exchange Rd[T] = saturate16(Rn[T] + Rm[B]) Rd[B] = saturate16(Rn[B] - Rm[T])

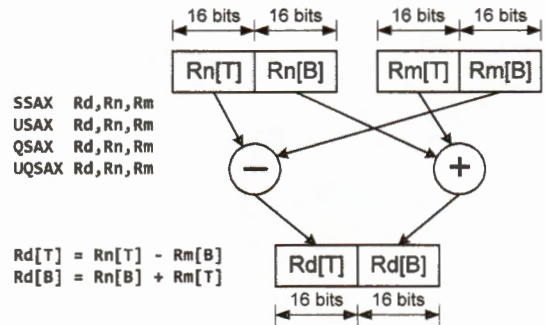
Table 24-8. Exchange halfwords of 2nd operand, add together top halfwords, and subtract between bottom halfwords

SSAX Rd,Rn,Rm	Signed subtract and add with exchange $Rd[T] = \text{truncate16}(Rn[T] - Rm[B])$ $Rd[B] = \text{truncate16}(Rn[B] + Rm[T])$
USAX Rd,Rn,Rm	Unsigned subtract and add with exchange $Rd[T] = \text{truncate16}(Rn[T] - Rm[B])$ $Rd[B] = \text{truncate16}(Rn[B] + Rm[T])$
QSAX Rd,Rn,Rm	Saturating subtract and add with exchange $Rd[T] = \text{saturate16}(Rn[T] - Rm[B])$ $Rd[B] = \text{saturate16}(Rn[B] + Rm[T])$
UQSAX Rd,Rn,Rm	Unsigned saturating subtract and add with exchange $Rd[T] = \text{saturate16}(Rn[T] - Rm[B])$ $Rd[B] = \text{saturate16}(Rn[B] + Rm[T])$

Table 24-9. Exchange halfwords of 2nd operand, subtract between top halfwords and add together bottom halfwords



ASX: Exchange 2nd operand, add and then subtract



SAX: Exchange 2nd operand, subtract and then add

Figure 24-14. Add and subtract with exchange (ASX), and Subtract and add with exchange (SAX)

The results can also be halved in parallel, as shown in Figure 24-15.

SHASX Rd,Rn,Rm	Signed halving add and subtract with exchange $Rd[T] = (Rn[T] + Rm[B])/2$ $Rd[B] = (Rn[B] - Rm[T])/2$
UHASX Rd,Rn,Rm	Unsigned halving add and subtract with exchange $Rd[T] = (Rn[T] + Rm[B])/2$ $Rd[B] = (Rn[B] - Rm[T])/2$
SHSAX Rd,Rn,Rm	Signed halving subtract and add with exchange $Rd[T] = (Rn[T] - Rm[B])/2$ $Rd[B] = (Rn[B] + Rm[T])/2$
UHSAX Rd,Rn,Rm	Unsigned halving subtract and add with exchange $Rd[T] = (Rn[T] - Rm[B])/2$ $Rd[B] = (Rn[B] + Rm[T])/2$

Table 24-10. Exchange, add/subtract (or subtract/add), and halving

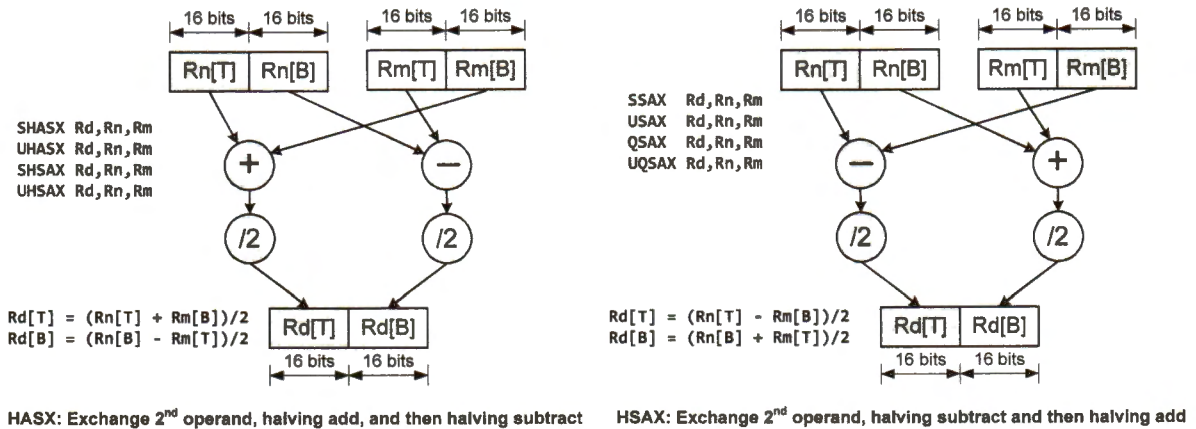


Figure 24-15. Exchange, add and subtract, and halving (ASXH), and Exchange, subtract and add, and halving (SAXH)

24.4.7 16-bit and 32-bit Multiplication

We can multiply two signed halfword integers selected from either the top or the bottom of two source operand registers. In these instructions, “T” specifies the top halfword of the corresponding operand, and “B” denotes the bottom halfword. The following lists instructions for signed halfword multiplication.

SMULBB Rd, Rn, Rm	Signed multiply bottom halfwords $Rd = Rn[B] * Rm[B]$
SMULBT Rd, Rn, Rm	Signed multiply bottom and top halfwords $Rd = Rn[B] * Rm[T]$
SMULTB Rd, Rn, Rm	Signed multiply top and bottom halfwords $Rd = Rn[T] * Rm[B]$
SMULTT Rd, Rn, Rm	Signed multiply top halfwords $Rd = Rn[T] * Rm[T]$

Table 24-11. Signed halfword multiplication

We can also multiply a 32-bit signed integer with a 16-bit signed integer selected from the top or bottom halfword of the second operand, as shown in Table 24-12. The actual product has 48 bits, but only the most significant 32 bits are kept.

SMULWB Rd, Rn, Rm	Signed multiply word by bottom halfword $Rd = (Rn * Rm[B]) \gg 16$
SMULWT Rd, Rn, Rm	Signed multiply word by top halfword $Rd = (Rn * Rm[T]) \gg 16$

Table 24-12. Signed multiplication between halfword and word

The SMULWB and SMULWT instructions can be used to implement conveniently signed fixed-point multiplication between Q15 and Q31, with the product kept in Q31 format.

We can also multiply two 32-bit signed integers, with the least significant 32 bits of the product truncated or rounded, as shown in Table 24-13. These two instructions multiply two Q31 fixed-point numbers, with the product remained in Q31 format. When the 64-bit exact product is reduced to 32 bits, the product can be either truncated or rounded to the nearest. The operation of rounding to the nearest is described in Chapter 12.2.6.

SMMUL Rd,Rn,Rm	Signed most significant word multiply with truncating $Rd = (Rn * Rm) \gg 32$
SMMULR Rd,Rn,Rm	Signed most significant word multiply with rounding to the nearest, $Rd = (Rn * Rm) \gg 32$

Table 24-13. Signed word multiplication

We can also multiply two pairs of 16-bit signed integers in parallel, calculate the sum or the difference of the 32-bit products, and store the result in the destination register, as shown in Figure 24-14. If there is a suffix X in the instruction mnemonic, the upper and lower halfword of the second source operand register is swapped before the multiplication operations are carried out. In other words, if the suffix X exists, the multiplication operations are *top* × *bottom* and *bottom* × *top*; otherwise, they are *top* × *top* and *bottom* × *bottom*.

SMUAD Rd,Rn,Rm	Signed dual multiply, then add products $Rd = Rn[B] * Rm[B] + Rn[T] * Rm[T]$
SMUADX Rd,Rn,Rm	Signed dual multiply with exchange, then add products $Rd = Rn[T] * Rm[B] + Rn[B] * Rm[T]$
SMUSD Rd,Rn,Rm	Signed dual multiply, then subtract products $Rd = Rn[B] * Rm[B] - Rn[T] * Rm[T]$
SMUSDX Rd,Rn,Rm	Signed dual multiply with exchange, then subtract products, $Rd = Rn[B] * Rm[T] - Rn[T] * Rm[B]$

Table 24-14. Dual 16-bit signed multiply with addition (or subtraction) of products

For Cortex-M3/M0, we can use the following C code to implement SMUAD instruction:

```
((x >> 16) * (y >> 16)) + ((x << 16) >> 16) * ((y << 16) >> 16))
```

where x and y are signed 32-bit integer variables, holding two Q15 values.

Although 16-bit multiplication does not generate overflow, the addition of two 32-bit products may overflow. SMUAD and SMUADX set the Q flag if the addition overflows.

```
LDR    r0, =0x7FFF7FFF
SMUAD  r3, r0, r0      ; r3=(215-1)×(215-1)+(215-1)×(215-1) = 0x7FFE0002, no overflow

LDR    r1, =0x80008000
SMUAD  r3, r1, r1      ; r3 = (-215)×(-215)+(-215)×(-215) = 231 = 0x80000000, overflow
SMUAD  r3, r0, r1      ; r3 = (-215)×(215-1)+(-215)×(215-1) = 0x80010000, overflow
```

SMUSD and SMUSDX do not affect the Q flag because no overflow can occur.

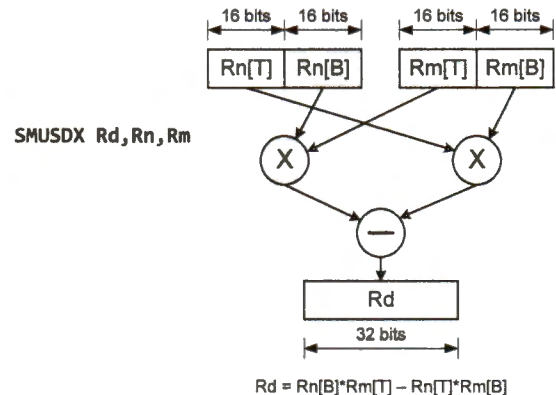
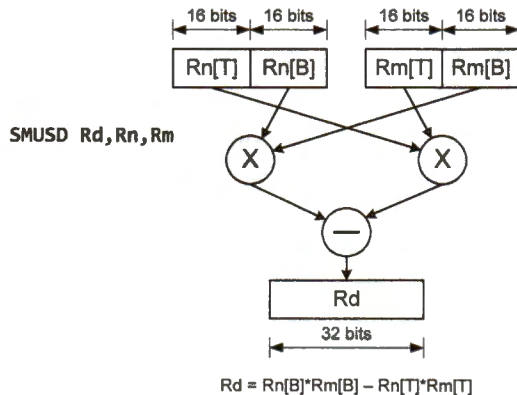
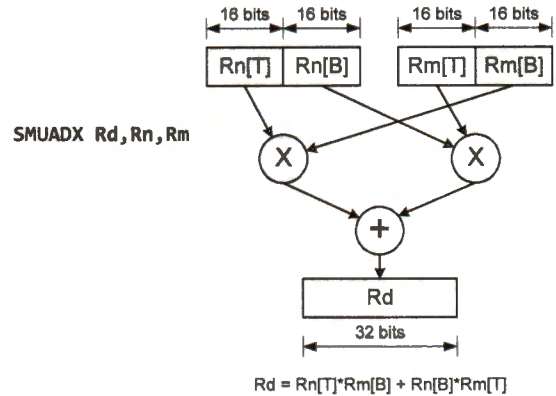
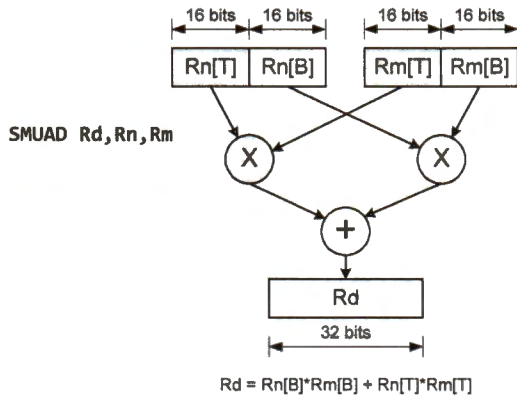


Table 24-15. Signed dual multiply add (or subtract)

One simple example is that we can use these instructions to calculate the sum or difference between two signed halfwords of a register.

```
MOV    r0, 0x00010001 ; top halfword = 1, bottom halfword = 1
SMUAD  r2, r1, r0      ; r2 = r1[T] + r1[B]
SMUSD  r2, r1, r0      ; r2 = r1[B] - r1[T]
SMUSDx r2, r1, r0      ; r2 = r1[T] - r1[B]
```

These instructions are very helpful to implement digital filtering in DSP. The current output $y(t)$ of a linear digital filter consists of a linear combination of previous M filter outputs, including $y(t-1)$, $y(t-2)$, ..., and $y(t-M)$, and N filter inputs including $x(t)$, $x(t-1)$, ..., and $y(t-N+1)$, as shown in the following equation:

$$y(t) = \sum_{i=0}^{N-1} a_i \cdot x(t-i) - \sum_{j=1}^M b_j \cdot y(t-j)$$

where a_i and b_j are coefficient constants, and N and M are pre-defined integers. N is referred to as the feed forward filter order, and M is known as the feedback filter order.

When all inputs are expressed in the Q15 format, these dual-multiplication instructions can be used to speed up the multiplication. If inputs are in the Q8 format, we need to extend the inputs to 16-bit signed numbers. Chapter 24.6 introduces parallel sign extension instructions.

24.4.8 16-bit Multiply and Accumulate with 64-bit Result

Multiply and Accumulate (MAC) is one of the key operations in DSP algorithms. For example, the output of a finite impulse response (FIR) filter, which removes all previous output items from the general form presented previously:

$$y(t) = \sum_{i=0}^{N-1} a_i \cdot x(t-i)$$

where N is the filter window length (or called the filter order), a_i is the filter's coefficients, and x is the sequence of input signal. This process slides over a fixed-length window of signal samples to compute a sequence of outputs $y(t)$.

In C, the finite impulse response (FIR) uses a loop to sum up the products repeatedly. Note that one MAC operation is performed in each loop.

```
while(1) {
    y[t] = 0; // Initialization
    for (i = 0, i < N; i++)
        y[t] = y[t] + a[i]*x[t-i]; // Multiply and accumulate
    t = t + 1;
}
```

To speed up the performance, microprocessors often have built-in hardware components to compute multiply and accumulate operations directly. Cortex-M4 and M7 provide MAC instructions with 16- or 32-bit input operands and 32- or 64-bit accumulator.

Figure 24-16, Figure 24-17, and Figure 24-18 present assembly instructions for signed 16-bit multiplication with a 64-bit accumulator. These instructions do not change the N, Z, C, V and Q flags in APSR.

SMLALD RdLo,RdHi,Rn,Rm	Dual signed multiply, accumulate long RdHi:RdLo = RdHi:RdLo + Rn[T]*Rm[T] + Rn[B]*Rm[B]
SMLSLD RdLo,RdHi,Rn,Rm	Dual signed multiply, subtract long RdHi:RdLo = RdHi:RdLo + Rn[T]* Rm[T] - Rn[B]*Rm[B]

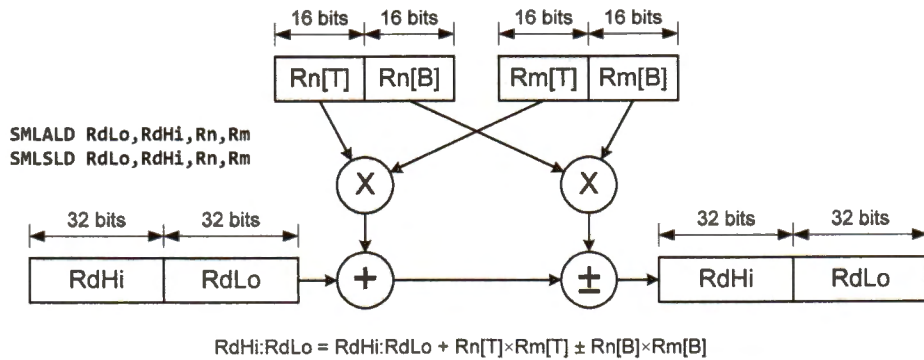


Figure 24-16. Dual 16-bit signed multiply, addition (or subtraction), and 64-bit accumulate.

The following shows a program that uses SMLALD to calculate the sum of squares. Given two arrays *A* and *B*, each of them has 1024 signed fixed-point numbers in Q15 format. We want to calculate the sum of squares defined as follows:

$$S = \sum_{i=0}^{1023} (a_i - b_i)^2$$

The following gives the assembly implementation. Because each fixed-point number has 16 bits, a load operation retrieves two Q15 values from the data memory and stores them in one register. After QSUB16 calculates the difference between two pairs of Q15 values, SMLALD performs the multiply and accumulate operations based on the difference obtained.

```

; The result is stored in registers r6:r5
LDR    r0, =A           ; Memory address of array A
LDR    r1, =B           ; Memory address of array B
MOV     r7, #256         ; Loop counter
MOV     r6, #0           ; Initialize upper word of sum
MOV     r5, #0           ; Initialize lower word of sum

loop LDR    r2, [r0], #4   ; Loads two halfwords from A, post-index
      LDR    r3, [r1], #4   ; Loads two halfwords from B, post-index
      QSUB16 r4, r2, r3     ; Two 16-bit subtractions in parallel
      SMLALD r5, r6, r4, r4 ; r6:r5 += r4[T]^2 + r4[B]^2
      SUBS   r7, r7, #1     ; Decrement loop counter
      BGT    loop

```

Example 24-10. Sum of squares

The following are dual 16-bit signed multiplication and 64-bit accumulation, with the halfwords of the second operand swapped before the multiplication.

SMLALDX RdLo,RdHi,Rn,Rm	Signed multiply accumulate long dual with exchange $RdHi:RdLo = RdHi:RdLo + Rn[T]*Rm[B] + Rn[B]*Rm[T]$
SMLSLDX RdLo,RdHi,Rn,Rm	Signed multiply subtract long dual with exchange $RdHi:RdLo = RdHi:RdLo + Rn[B]*Rm[T] - Rn[T]*Rm[B]$

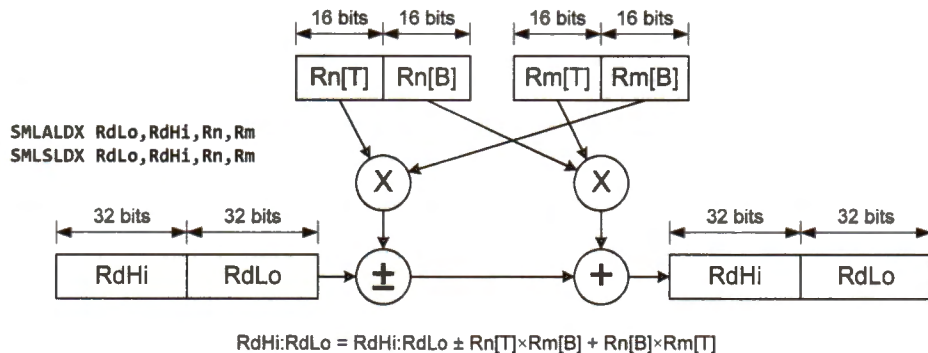


Figure 24-17. Exchange, dual signed multiply, addition/subtraction, and 64-bit accumulate.

The following shows single 16-bit signed multiplication and 64-bit accumulation.

SMLALBB RdLo,RdHi,Rn,Rm	$RdHi:RdLo = RdHi:RdLo + Rn[B]*Rm[B]$
SMLALBT RdLo,RdHi,Rn,Rm	$RdHi:RdLo = RdHi:RdLo + Rn[B]*Rm[T]$
SMLATTB RdLo,RdHi,Rn,Rm	$RdHi:RdLo = RdHi:RdLo + Rn[T]*Rm[B]$
SMLALTT RdLo,RdHi,Rn,Rm	$RdHi:RdLo = RdHi:RdLo + Rn[T]*Rm[T]$

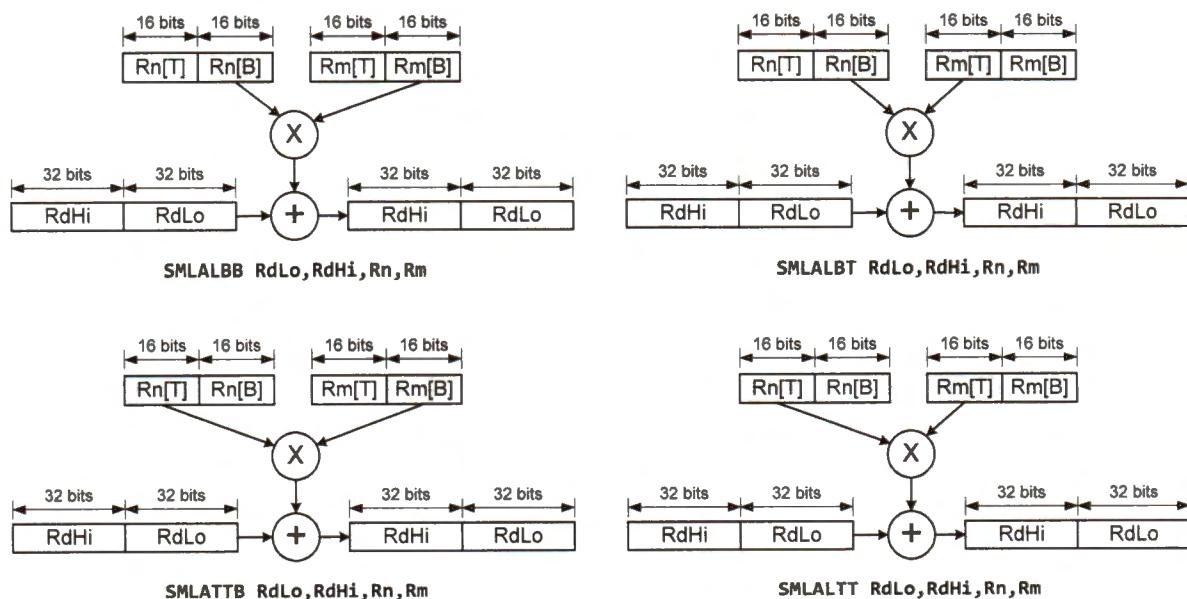


Figure 24-18. 16-bit signed multiply with 64-bit accumulate

24.4.9 16-bit Multiply and Accumulate with 32-bit Result

The accumulator can also be 32 bits in the MAC instructions discussed previously. Figure 24-19, Figure 24-20, and Figure 24-21 shows detailed operations. No overflow can occur during the 16-bit multiplication. However, overflow may occur during the accumulation. SMLAD and SMLSD set the Q flag in APSR if overflow occurs.

SMLAD Rd,Rn,Rm,Ra	Signed multiply accumulate dual $Rd = Ra + Rn[T] * Rm[T] + Rn[B] * Rm[B]$
SMLSD Rd,Rn,Rm,Ra	Signed multiply subtract dual $Rd = Ra + Rn[B] * Rm[B] - Rn[T] * Rm[T]$

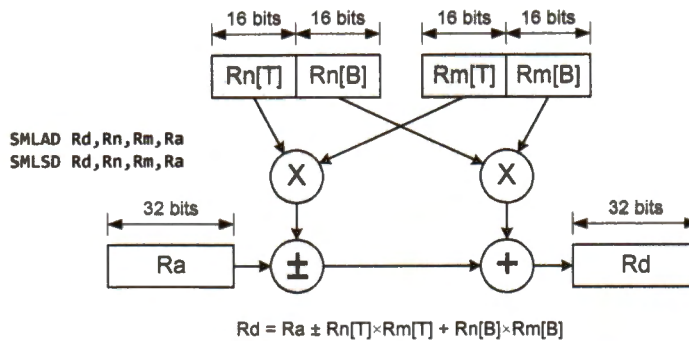


Figure 24-19. Dual 16-bit signed multiply with 32-bit accumulate (or subtract)

The top and bottom halfwords of the second source operand can be swapped before the multiplication if the suffix X is appended. The multiplication is then *top* $\times$ *bottom* and *bottom* $\times$ *top*. The Q flag is set if the addition overflows.

SMLADX Rd,Rn,Rm,Ra	Signed multiply accumulate dual with exchange $Rd = Ra + Rn[T] * Rm[B] + Rn[B] * Rm[T]$
SMLSDX Rd,Rn,Rm,Ra	Signed multiply subtract dual with exchange $Rd = Ra + Rn[B] * Rm[T] - Rn[T] * Rm[B]$

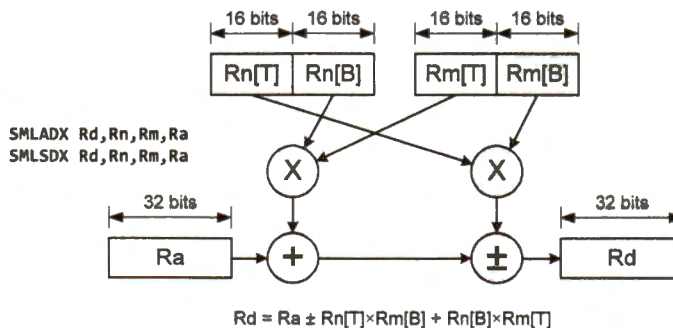


Figure 24-20. Exchange, dual 16-bit multiply, and 32-bit accumulate (or subtract)

We can also multiply two specific 16-bit signed values, either from the top or bottom halfword from two registers, and add the 32-bit signed product to an accumulator register. The Q flag is set if the accumulation overflows.

SMLABB Rd,Rn,Rm,Ra	Signed multiply bottom, accumulate long $Rd = Ra + Rn[B] * Rm[B]$
SMLABT Rd,Rn,Rm,Ra	Signed multiply bottom and top, accumulate long $Rd = Ra + Rn[B] * Rm[T]$
SMLATB Rd,Rn,Rm,Ra	Signed multiply top and bottom, accumulate long $Rd = Ra + Rn[T] * Rm[B]$
SMLATT Rd,Rn,Rm,Ra	Signed multiply top, accumulate long $Rd = Ra + Rn[T] * Rm[T]$

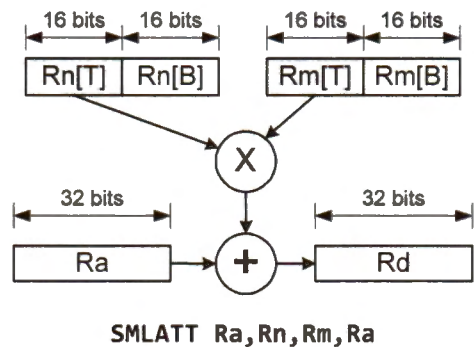
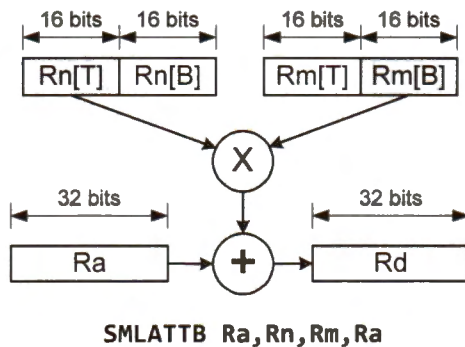
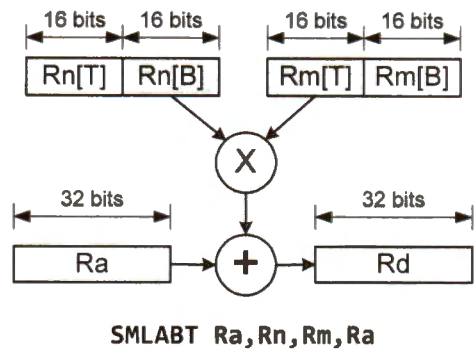
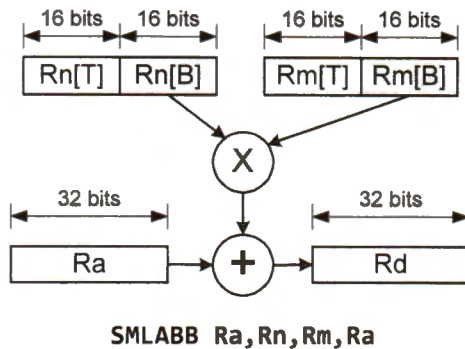


Figure 24-21. Single 16-bit multiply with 32-bit accumulate

The following assembly code can be used in a loop to calculate $\sum_{i=0}^n a_i \cdot x_i$, in which a_i and x_i are 16-bit signed values, such as Q15.

```

LDR    r4, [r1], #4    ; load a[i], a[i+1], r1 points to a[i+2] after loading
LDR    r3, [r0], #4    ; load x[i], x[i+1], r0 points to x[i+2] after loading
SMLABB r5, r3, r4, r5   ; sum = x[i]*a[i], r5 = sum
SMLATT r5, r3, r4, r5   ; sum = x[i+1]*a[i+1], r5 = sum

```


24.4.10 16×32 Multiply and Accumulate with 32-bit Result

We can also multiply the bottom or top halfword of one register with another register, with the product accumulated. If overflow occurs upon accumulating the product, the Q flag bit in APSR is set. The multiplication in SMLAWB and SMLAWT cannot cause overflow.

SMLAWB Rd,Rn,Rm,Ra	Signed multiply word by bottom halfword, accumulate $Rd = Ra + (Rn * Rm[B]) \gg 16$
SMLAWT Rd,Rn,Rm,Ra	Signed multiply word by top halfword, accumulate $Rd = Ra + (Rn * Rm[T]) \gg 16$

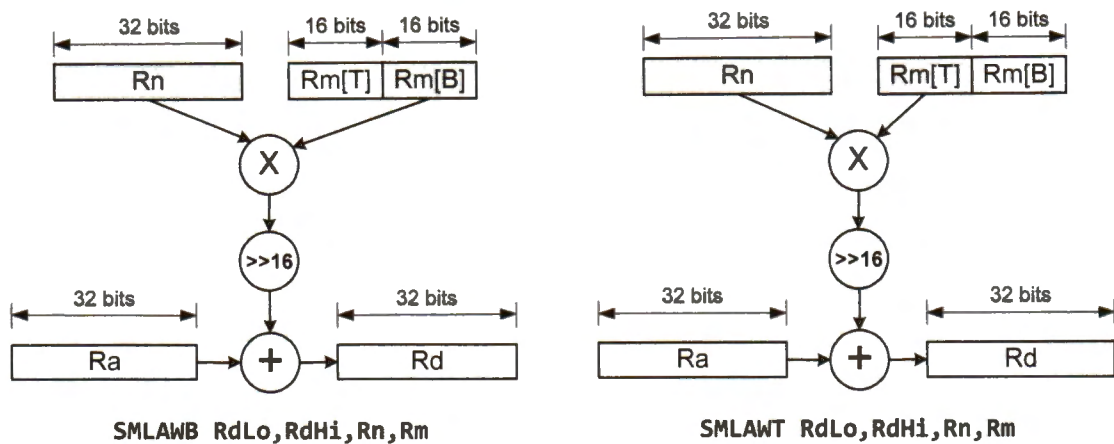


Figure 24-22. Single 16×32 multiply with 32-bit accumulate

24.4.11 32×32 Multiply and Accumulate with 32-bit Result

SMLAL and UMLAL are two instructions that first multiply two words and then accumulate the product to a 64-bit sum, as introduced in Chapter 4.4.

Cortex-M4 and M7 also support 32-bit accumulation with different rounding schemes. If R is specified, it uses rounding to nearest; otherwise, it uses rounding towards zero (*i.e.*, truncation).

SMMLA Rd,Rn,Rm,Ra	Signed most significant word multiply accumulate, $Rd = Ra + (Rn * Rm) \gg 32$, Truncate
SMMLAR Rd,Rn,Rm,Ra	Signed most significant word multiply accumulate, $Rd = Ra + (Rn * Rm) \gg 32$, Round to nearest
SMMLS Rd,Rn,Rm,Ra	Signed most significant word multiply subtract, $Rd = Ra - (Rn * Rm) \gg 32$, Truncate
SMMLSR Rd,Rn,Rm,Ra	Signed most significant word multiply subtract, $Rd = Ra - (Rn * Rm) \gg 32$, Round to nearest

24.4.12 Unsigned Long Multiply with Accumulate Accumulate

A special instruction UMAAL is implemented in hardware. It can be used to speed up the Separated Operand Scanning (SOS) algorithm (see reference [92] and [93]). The SOS algorithm multiplies two very large unsigned integers, such as 256 bits by 256 bits. UMAAL performs accumulation twice after the multiplication. It adds the upper word and the lower word of the 64-bit accumulator to the accumulator.

UMAAL RdLo,RdHi,Rn,Rm	Unsigned multiply accumulate accumulate long RdHi:RdLo = Rn*Rm + RdHi + RdLo
------------------------------	---

Suppose we want to multiply two large unsigned numbers A and B . Each of them has m words, *i.e.*, $A = A_{m-1} \cdots A_2A_1A_0$ and $B = B_{m-1} \cdots B_2B_1B_0$. Both A_i and B_i have 32 bits ($0 \leq i \leq m - 1$), and A_0 and B_0 are the least significant word.

Let $P = A \times B$ and P has a total of $2m$ words. The SOS algorithm works similarly to standard pencil-and-paper multiplication. Figure 24-23 shows the procedure of multiplying two double-word integers A and B . Differing from standard multiplication, SOS uses block-based multiplications.

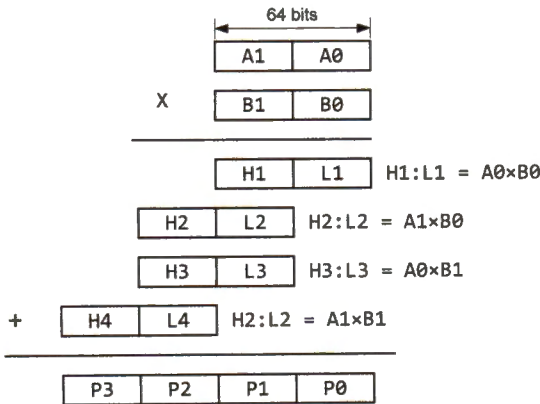


Figure 24-23. Multiplying two unsigned 64-bit integers

```
for i = 0 to 2m-1
    P[i] ← 0;           // Initialization

for i = 0 to m-1
    C ← 0
    for j = 0 to m-1
        C:S ← A[j]*B[i] + C + P[i+j]
        P[i+j] ← S
    P[i+m] ← C
```

Example 24-11. Pseudocode for Separated Operand Scanning (SOS) algorithm

The SOS algorithm, with pseudocode given in Example 24-11, performs two accumulation operations after the multiplication. The multiplication result consists of $2m$ words, including $P[0], P[1], \dots, P[2m - 1]$, with the least significant word in $P[0]$. The inner loop performs multiply-accumulate-accumulate operations in the following form:

$$C:S = a \times b + C + S$$

where the size of a , b , C and S is a word. The UMAAL instruction performs such operation.

SOS calculates the product P starting from the least significant word. The inner loop computes and accumulates partial product. It reuses the results stored in the 64-bit accumulator ($C:S$) for the next partial product.

During the computation process, we need to change the value of the upper or lower word of the accumulator before executing UMAAL. Therefore, the lower word (S) should be saved to the memory within the inner loop and the upper word (C) should be saved to the memory within the outer loop.

Figure 24-24 shows the basic steps of multiplying two 64-bit unsigned integers. The final product consists of 128 bits: $P[3] = C4$, $P[2] = S4$, $P[1] = S3$, and $P[0] = S1$. This example executes UMAAL four times. The second UMAAL execution does not directly produce any full result. However, it produces partial results, $C2$ and $S4$, which are used when calculating $P[2]$ and $P[1]$.

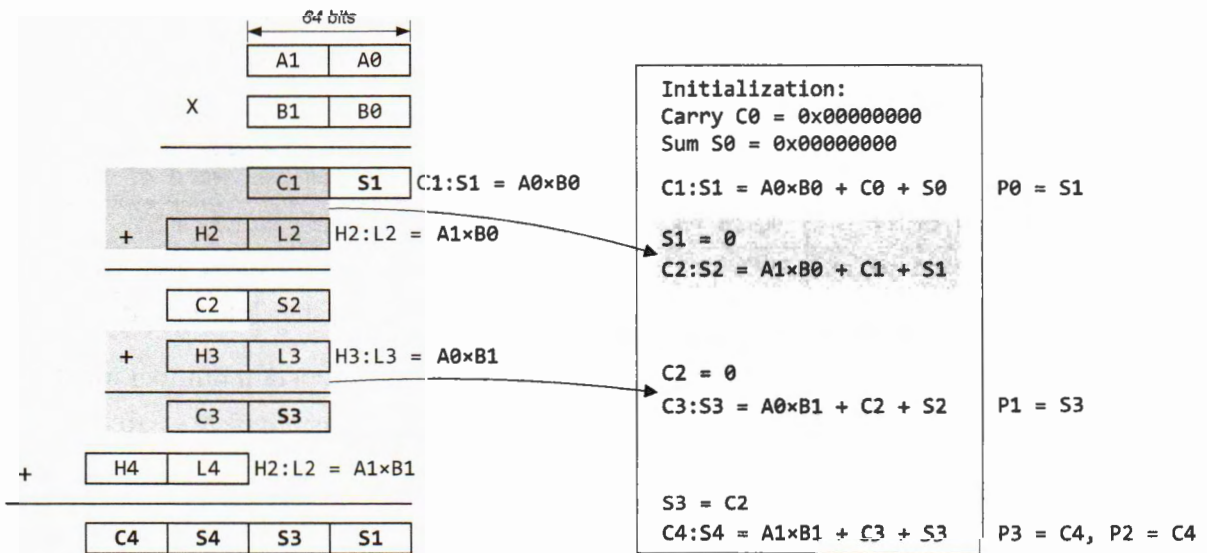


Figure 24-24. Accelerating SOS by using UMAAL

24.5 Packing Halfwords into a Word

We can pack two halfwords from two different registers into another register, as shown in Figure 24-25.

PKHBT Rd,Rn,Rm{,Op2}	Pack halfword. Rd = Rn[B]:(Rm,Op2)[T] Op2 = LSL #n, n ≠ 0
PKHTB Rd,Rn,Rm{,Op2}	Pack halfword. Rd = Rn[T]:(Rm,Op2)[B] Op2 = ASR #n, n ≠ 0

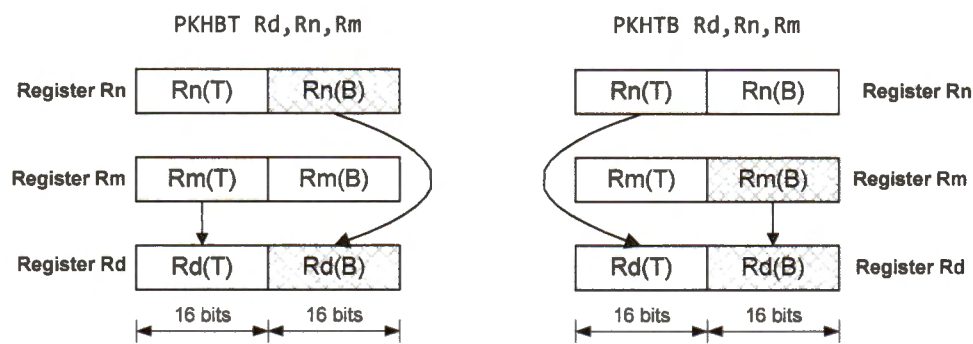


Figure 24-25. Packing two halfwords into a word (Bottom + Top, or Top + Bottom)

The following table gives the assembly instructions of four possible halfword packing operations. When packing two top halfwords, the second source operand must be shifted right by 16 bits. When packing two bottom halfwords, the second source operand should be shifted left by 16 bits. This is because ASR is not permitted for the second source operand in PKHBT, and LSL is not allowed in PKHTB.

	Top halfword of r1	Bottom halfword of r1
Top halfword of r0	PKHTB r3, r0, r1, ASR #16	PKHTB r3, r0, r1
Bottom halfword of r0	PKHBT r3, r0, r1	PKHBT r3, r0, r1, LSL #16

Example 24-12. Packing halfwords from two different registers into a word

To facilitate 16-bit SIMD instructions, we often need to place a constant number into both halfwords of a register. The following example code adds a constant 0x1234 to both the upper and lower halfword in the r4 register.

```

MOV    r3, #0x1234           ; r3 = 0x1234
PKHBT  r5, r3, r3, LSL #16   ; r5 = 0x12341234
SADD16 r4, r4, r5            ; add 0x1234 to both halfwords of r4

```


24.6 Signed and Unsigned Extension

As introduced in Chapter 4.8, SXTB, UXTB, SXTH, UXTH can extend a signed or unsigned byte or halfword to a word.

Cortex-M4 and M7 add two SIMD instructions, SXTB16 and UXTB16, which extend two bytes to two halfwords in parallel, as shown in Figure 24-26.

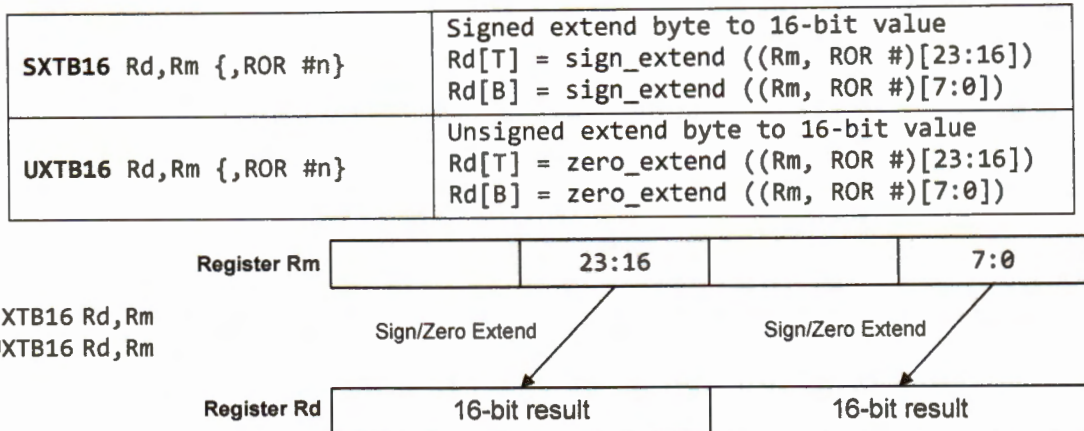


Figure 24-26. Signed and unsigned parallel extension of two bytes to two halfwords

Cortex-M4 and M7 also provide instructions for signed/unsigned extension and then addition, as listed below. The operations can be applied to a byte or halfword, or simultaneously to two bytes or halfwords.

SXTAB Rd,Rn,Rm{,ROR #}	Signed extend byte to word, and add $Rd = Rn + \text{sign_extend}((Rm, \text{ROR \#})[7:0])$
UXTAB Rd,Rn,Rm{,ROR #}	Unsigned extend byte to word, and add $Rd = Rn + \text{zero_extend}((Rm, \text{ROR \#})[7:0])$
SXTAH Rd,Rn,Rm{,ROR #}	Signed extend halfword to word, and add $Rd = Rn + \text{sign_extend}((Rm, \text{ROR \#})[15:0])$
UXTAH Rd,Rn,Rm{,ROR #}	Unsigned extend halfword to word, and add $Rd = Rn + \text{zero_extend}((Rm, \text{ROR \#})[15:0])$
SXTAB16 Rd,Rn,Rm{,ROR #}	Signed extend 2 bytes to 2 halfwords, and add $Rd[T] = Rn[T] + \text{sign_extend}((Rm, \text{ROR \#})[23:16])$ $Rd[B] = Rn[B] + \text{sign_extend}((Rm, \text{ROR \#})[7:0])$
UXTAB16 Rd,Rn,Rm{,ROR #}	Unsigned extend 2 bytes to 2 halfwords, and add $Rd[T] = Rn[T] + \text{zero_extend}((Rm, \text{ROR \#})[23:16])$ $Rd[B] = Rn[B] + \text{zero_extend}((Rm, \text{ROR \#})[7:0])$

Table 24-16. Instructions for extension and add

24.7 GE Flags

Most SIMD instructions do not change the N, Z, C, V and Q flags of the Application Program Status Register (APSR). However, there are SIMD instructions, as listed in Table 24-17, which can change the GE (Greater than or Equal) flags in APSR.

Table 24-17 summarizes the setting of GE flags for SIMD instructions. None of them is saturating instructions.

	GE = 1	GE = 0
SADD8, SADD16	The corresponding result is greater than or equal to zero.	The corresponding result is smaller than zero.
SSUB8, SSUB16		
SASX, SSAX		
UADD8 UADD16	The corresponding result did overflow, generating a carry.	The corresponding result did not overflow.
USUB8 USUB16	The corresponding result is greater than or equal to zero, meaning borrow did not occur.	The corresponding result is smaller than zero, meaning borrow did occur.
UASX USAX	1. “A” (Add) sets GE flags in the same way as ADDS sets the C flag. If overflow occurs, the corresponding GE flags are set. 2. “S” (Subtract) sets GE flags in the same way as SUBS sets the C flag. If borrow occurs, the corresponding GE flags are cleared.	

Table 24-17. Instructions that affect the GE flags in APSR

The GE flags are mostly used by the SEL instruction for selecting data from either the first or the second source operand register, as introduced in the next section.

- For signed arithmetic operations, including SADD8, SADD16, SSUB8, SSUB16, SASX, and SSAX, the GE flag is set if the corresponding result is greater than or equal to zero.
- For unsigned arithmetic operations, including UADD8, UADD16, USUB8, USUB16, UASX, and USAX, the GE flag is set if overflow occurs on unsigned addition or if no borrow occurs on unsigned subtraction.

The GE flags have four bits, one flag bit for each 8-bit parallel add or subtract, or two flag bits for each 16-bit parallel add or subtract.

For parallel 8-bit arithmetic operations listed in Table 24-17,

- flag GE[0] is for bits[7:0] of the result,
- flag GE[1] is for bits[15:8] of the result,
- flag GE[2] is for bits[23:16] of the result, and
- flag GE[3] is for bits[31:24] of the result.

For parallel 16-bit arithmetic operations listed above,

- flags GE[1:0] are for bits[15:0] of the result, and
- flags GE[3:2] are for bits[31:16] of the result.

For example, both GE[1] and GE[0] are set if the lower halfword result of SADD16 is greater than or equal to zero.

24.8 Byte Selection Instruction

In Chapter 4.11, we have introduced the instructions UBFX and SBFX to extract bits from a word. Cortex-M4 and M7 provide instruction SEL, which selects bytes or halfwords from two source operand registers depending on the GE flags in APSR, as shown in Figure 24-27.

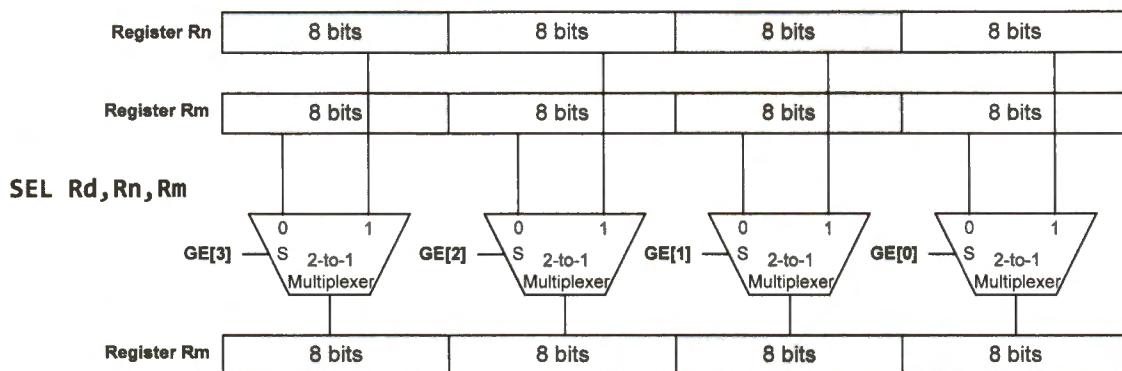


Figure 24-27. Select bytes from Rn or Rm based on GE flag bits in APSR

For example, we can use the SEL instructions to find the pairwise max and min between four pairs of Q7 values.

```
SSUB8 r3, r0, r1    ; If borrow does not occur, corresponding GE flag is 1
                    ; i.e., if result ≥ 0, GE = 1

; Select the larger Q7 of each pair
SEL    r4, r0, r1    ; if GE is 1, select byte from r0; otherwise from r1

; Select the smaller Q7 of each pair
SEL    r5, r1, r0    ; if GE is 1, select byte from r1; otherwise from r0
```

Example 24-13. Select the max and min of the corresponding signed bytes of r0 and r1

For parallel halfword arithmetic operations, two GE bits are set for each pair of halfword operation. Two bit flags GE[1:0] are for the lower halfword operation, and GE[2:0] are for the upper halfword operation. A halfword operation sets its both flag bits to the same value. Therefore, SEL instruction can also use the GE flags to select halfwords.

```
SSUB16 r3, r0, r1 ; If result ≥ 0, corresponding GE flag is 1
SEL     r4, r0, r1 ; Select the larger Q15 of each pair
SEL     r5, r1, r0 ; Select the smaller Q15 of each pair
```

Example 24-14. Choose the max and min of the corresponding signed halfword of r0 and r1

24.9 Basic DSP Functions

In this section, we focus on Q7 and Q15 basic vector-based operations, including

- Vector negate
- Vector absolute value
- Vector offset
- Vector shift left
- Vector mean
- Vector multiplication
- Vector dot product
- Vector min and max

To simplify assembly programs, we assume that the number of values is a multiple of four if the input arrays hold Q7 numbers and a multiple of two if the input arrays hold Q15 numbers.

24.9.1 Vector Negate

The following assembly program reverses the sign of all numbers in a Q7 array. It takes three arguments, including a pointer to the source array, a pointer to the destination array, and the total number of Q7 values in the source array. The program uses SIMD instruction QSUB8 to negate four Q7 values in parallel by subtracting them from zero.

```
vector_negate_Q7 PROC
    EXPORT vector_negate_Q7

    ; r0 = pSrc, pointer to source Q7 array
    ; r1 = pDst, pointer to destination Q7 array
    ; r2 = # of Q7 values in source array

    PUSH    {r4,r5,lr}
    MOV     r3, #0
```



```

MOV    r4, #0          ; Initialize loop counter

B      check

loop   LDR    r5, [r0], #4    ; Load four Q7 values, post-index
       QSUB8  r5, r3, r5      ; Negate four Q7 values in parallel
       STR    r5, [r1], #4    ; Store results to destination array, post-index
       ADDS   r4, r4, #4      ; Processed four Q7 values in each loop
check  CMP    r4, r2
       BLT    loop

POP    {r4,r5,pc}

ENDP

```

Example 24-15. Negate an array of Q7 numbers

The following program uses QSUB16 instructions to reverse the sign of two Q16 numbers in parallel.

```

vector_negate_Q15 PROC
EXPORT vector_negate_Q15

; r0 = pSrc, pointer to source Q15 array
; r1 = pDst, pointer to destination Q15 array
; r2 = # of Q15 values in source array

PUSH   {r4,r5,lr}
MOV    r3, #0
MOV    r4, #0          ; Initialize loop counter
B      check

loop   LDR    r5, [r0], #4    ; Load two Q15 values, post-index
       QSUB16 r5, r3, r5      ; Negate two Q15 values in parallel
       STR    r5, [r1], #4    ; Store two Q15 values, post-index
       ADDS   r4, r4, #2      ; Increase loop counter
check  CMP    r4, r2          ; Compare with array size
       BLT    loop

POP    {r4,r5,pc}

ENDP

```

Example 24-16. Negate an array of Q15 numbers

24.9.2 Vector Absolution Value

We can use SSUB8 and SEL to find the absolute values of all Q7 numbers in an array. The program processes four Q7 numbers in each loop. It first finds all positive values, then

negates original numbers and finds positive ones again, and finally merges all positive values into the destination register.

```

vector_abs_Q7 PROC
    EXPORT    vector_abs_Q7

    ; r0 = pSrc, pointer to source Q7 array
    ; r1 = pDst, pointer to destination Q7 array
    ; r2 = # of Q7 values in source array

    PUSH    {r4-r8, LR}        ; Preserve registers
    MOV     r6, #0
    MOV     r3, #0              ; Initialize loop counter
    B       check

loop  LDR     r4, [r0], #4      ; Load four Q7 values

    SSUB8   r5, r4, r6          ; Set four GE bit flags, r6 = 0
    SEL     r7, r5, r6          ; Select all positive Q7 based on GE flags

    SSUB8   r5, r6, r4          ; Negate r4, and set four GE flags
    SEL     r8, r5, r6          ; Select all positive Q7 based on GE flags

    ORR     r5, r7, r8          ; Merge two selections
    STR     r5, [r1], #4        ; Store results

    ADDS    r3, r3, #4          ; Increase loop counter
check CMP    r3, r2             ; Compare with array size
    BLT     loop

    POP     {r4-r8, pc}

    ENDP

```

Example 24-17. Find the absolute values of a Q7 array

Similar operations can also be performed for Q15 values. The following program stores the absolute value of a Q15 array to the destination array. Within each loop, two Q15 numbers are processed in parallel.

```

vector_abs_Q15 PROC
    EXPORT    vector_abs_Q15

    ; r0 = pSrc, pointer to source Q15 array
    ; r1 = pDst, pointer to destination Q15 array
    ; r2 = # of Q15 values in source array

    PUSH    {r4-r8, LR}        ; Preserve registers
    MOV     r6, #0

```

```

MOV    r3, #0           ; Initialize loop counter
B      check

loop   LDR    r4, [r0], #4       ; Load two Q15 values

      SSUB16 r5, r4, r6         ; Set GE flags
      SEL    r7, r5, r6         ; Select all positive Q15 based on GE flags

      SSUB16 r5, r6, r4         ; Negate r4, and set GE flags
      SEL    r8, r5, r6         ; Select all positive Q15 based on GE flags

      ORR    r5, r7, r8         ; Merge two selections
      STR    r5, [r1], #4       ; Store results

      ADDS   r3, r3, #2         ; Increase loop counter
check  CMP    r3, r2             ; Compare with array size
      BLT    loop

      POP    {r4-r8, pc}

      ENDP

```

Example 24-18. Find the absolute values of a Q15 array

24.9.3 Vector Offset with Saturation

The following assembly program shows how to add a constant Q7 offset to all Q7 numbers in an array. The program packs the 8-bit input *offset* into the 32-bit format *offset:offset:offset:offset*, and then uses QADD8 to add simultaneously four pairs of 8-bit Q7 numbers with saturation.

The program uses UXTB to zero extend the 8-bit *offset* input to 32 bits (*i.e.*, making the most significant 24 bits as zero). This is a preventive operation in case the input *offset* has been sign-extended to 32 bits when it is passed to this function via register r3.

```

vector_offset_Q7 PROC
EXPORT vector_offset_Q7

; r0 = pSrc, pointer to source Q7 array
; r1 = pDst, pointer to destination Q7 array
; r2 = # of Q7 values in source array
; r3 = Offset constant in Q7 format

PUSH    {r4-r6, lr}

; Pack 8-bit offset to 32 bits to use SIMD addition
MOV     r5, #0xFF00
UXTB    r6, r3           ; r6 = zero extend offset to 32 bits
AND     r5, r5, r3, LSL #8 ; r5 = 00:00:offset:00

```

```

    ORR    r6, r6, r5          ; r6 = 00:00:offset:offset
    MOV    r5, #0xFF0000
    AND    r5, r5, r3, LSL #16 ; r5 = 00:offset:00:00
    ORR    r6, r6, r5          ; r6 = 00:offset:offset:offset
    ORR    r6, r6, r3, LSL #24 ; r6 = offset:offset:offset:offset

    MOV    r4, #0              ; Initialize loop counter

    B      check

loop    LDR    r5, [r0], #4      ; Load four Q7 values, post-index
        QADD8  r5, r5, r6      ; Add four pairs of Q7 values
        STR    r5, [r1], #4    ; Store four Q7 values, post-index
        ADDS   r4, r4, #4      ; Increase loop counter
check   CMP    r4, r2          ; Compare with array size
        BLT    loop

    POP    {r4-r6, pc}
    ENDP

```

Example 24-19. Adding a constant offset to a Q7 array

The following example adds an offset to all values in a Q15 array. The program runs the PKHBT instruction to pack the 16-bit input *offset* into the format *offset:offset* to use QADD16.

```

vector_offset_Q15 PROC
    EXPORT vector_offset_Q15

    ; r0 = pSrc, pointer to source Q15 array
    ; r1 = pDst, pointer to destination Q15 array
    ; r2 = # of Q15 values in source array
    ; r3 = offset in Q15 format

    PUSH    {r4-r6, lr}
    PKHBT   r5, r3, r3, LSL #16 ; r5 = offset:offset
    MOV     r4, #0              ; Initialize loop counter

    B      check

loop    LDR    r6, [r0], #4      ; Load two Q15 values, post-index
        QADD16 r6, r6, r5      ; Add two pairs of Q15 values
        STR    r6, [r1], #4    ; Store two Q15 values, post-index
        ADDS   r4, r4, #2      ; Increase loop counter
check   CMP    r4, r2          ; Compare with array size
        BLT    loop

    POP     {r4-r6, pc}
    ENDP

```

Example 24-20. Adding a constant offset to a Q15 array

24.9.4 Vector Shift Left with Saturation

The following program shifts left Q7 numbers with saturation. For example, with saturation, $0x66 \ll 1 = 0x7F$.

```

vector_shift_left_Q7 PROC
    EXPORT vector_shift_left_Q7

    ; r0 = pSrc, pointer to source Q7 array
    ; r1 = pDst, pointer to destination Q7 array
    ; r2 = # of Q7 values in source array
    ; r3 = # of bits to be shifted

    PUSH    {r4-r10, lr}
    MOV     r5, #0xFF00
    MOV     r6, #0xFF0000
    MOV     r4, #0                ; Initialize loop counter
    B       check

loop    LDRSB r7, [r0], #1        ; Load 1st Q7, sign extend to a word
        LDRSB r8, [r0], #1        ; Load 2nd Q7, sign extend to a word
        LDRSB r9, [r0], #1        ; Load 3rd Q7, sign extend to a word
        LDRSB r10, [r0], #1       ; Load 4th Q7, sign extend to a word

        LSL    r7, r7, r3          ; Logic shift left 1st Q7 by r3 bits
        SSAT   r7, #8, r7          ; Saturate as 8-bit value
        AND    r7, r7, #0xFF       ; Keep only lowest 8 bits

        LSL    r8, r8, r3          ; Logic shift left 2nd Q7 by r3 bits
        SSAT   r8, #8, r8          ; Saturate as 8-bit value
        AND    r8, r5, r8, LSL #8  ; Shift result to position [15:8]
        ORR    r7, r7, r8          ; Combine the result of 1st and 2nd Q7

        LSL    r8, r9, r3          ; Logic shift left 3rd Q7 by r3 bits
        SSAT   r8, #8, r8          ; Saturate as 8-bit value
        AND    r8, r6, r8, LSL #16 ; Shift result to position [23:16]
        ORR    r7, r7, r8          ; Combine the result of 3rd Q7

        LSL    r8, r10, r3         ; Logic shift left 4th Q7 by r3 bits
        SSAT   r8, #8, r8          ; Saturate as 8-bit value
        ORR    r7, r7, r8, LSL #24 ; Combine the result of 4th Q7

        STR    r7, [r1], #4        ; Store result to destination array

        ADDS   r4, r4, #4          ; Increase loop counter
check   CMP     r4, r2              ; Compare with array size
        BLT    loop
        POP     {r4-r10, pc}
    ENDP

```

Example 24-21. Shifting left all values of a Q15 array

The following program shifts left Q15 numbers with saturation.

```

vector_shift_left_Q15 PROC
    EXPORT vector_shift_left_Q15

    ; r0 = pSrc, pointer to source Q15 array
    ; r1 = pDst, pointer to destination Q15 array
    ; r2 = # of Q15 values in source array
    ; r3 = # of bits to be shifted

    PUSH    {r5-r6, lr}
    MOV     r4, #0                ; Initialize loop counter
    B       check

loop    LDRSH    r6, [r0], #2        ; Load 1st Q15, sign extend to a word
        LSLS     r6, r6, r3        ; Logic shift left 1st Q15 by r3 bits
        SSAT     r6, #16, r6        ; Saturated as 16-bit value

        LDRSH    r5, [r0], #2        ; Load 2nd Q15, sign extend to a word
        LSLS     r5, r5, r3        ; Logic shift left 2nd Q15 by r3 bits
        SSAT     r5, #16, r5        ; Saturated as 16-bit value
        PKHBT    r5, r6, r5, LSL #16 ; r5 = r6[15:0]:r5[15:0]
        STR      r5, [r1], #4        ; Store result to destination array

        ADDS     r4, r4, #2          ; Increase loop counter
check   CMP      r4, r2              ; Compare with array size
        BLT      loop
        POP      {r5-r6, pc}

    ENDP

```

Example 24-22. Shifting left all values of a Q15 array

24.9.5 Vector Mean

The following program calculates the arithmetic mean of an array of Q7 values, defined by the formula below:

$$\bar{a} = \frac{1}{n} \sum_{i=0}^{n-1} a_i$$

The program extracts individual bytes and adds them up. SXTAB extracts the least significant byte of a register, and extends the byte to 32-bit signed number, and adds the 32-bit integer with another register.

```

vector_mean_Q7 PROC
    EXPORT vector_mean_Q7
    ; r0 = pSrc, pointer to source Q7 array
    ; r1 = # of Q7 values in source array

```

```

        PUSH    {r4-r6, lr}
        MOV     r3, #0                ; sum = 0
        MOV     r4, #0                ; Initialize loop counter
        B       check

loop    LDR     r5, [r0], #4          ; Load four Q7 values, post-index
        SXTAB  r3, r3, r5            ; sum += in[7:0]
        SBFX   r6, r5, #8, #8       ; r6 = in[15:8]
        ADD    r3, r3, r6            ; sum += in[15:8]
        SBFX   r6, r5, #16, #8      ; r6 = in[23:16]
        ADD    r3, r3, r6            ; sum += in[23:16]
        ADD    r3, r3, r5, ASR #24  ; sum += in[31:24]
        ADDS   r4, r4, #4            ; Increase loop counter
check   CMP    r4, r1                ; Compare with array size
        BLT    loop

        SDIV   r3, r3, r1
        MOV    r0, r3                ; sum returned in r0
        POP    {r4-r6, pc}
        ENDP

```

Example 24-23. Calculating the arithmetic mean of a Q7 array

The following program uses the MAC instruction SMLAD to speed up the calculation of the arithmetic mean.

```

vector_mean_Q15 PROC
    EXPORT vector_mean_Q15
    ; r0 = pSrc, pointer to source Q15 array
    ; r1 = # of Q15 values in source array

    PUSH    {r4-r6, lr}
    MOV     r3, #0                ; sum = 0
    MOV     r6, #0x00010001       ; r3[T] = 1, r3[B] = 1
    MOV     r4, #0                ; Initialize loop counter

    B       check

loop    LDR     r5, [r0], #4          ; Load four Q15 values, post-index
        SMLAD  r3, r5, r6, r3        ; r3 = 1*r5[T] + 1*r5[B]
        ADDS   r4, r4, #2            ; Increase loop counter
check   CMP    r4, r1                ; Compare with array size
        BLT    loop

        SDIV   r3, r3, r1
        MOV    r0, r3                ; sum returned in r0
        POP    {r4-r6, pc}
        ENDP

```

Example 24-24. Calculating the arithmetic mean of a Q15 array

24.9.6 Vector Pairwise Multiplication

Given two arrays, a_i and b_i ($0 \leq i \leq n-1$), the following programs calculate the pairwise product for Q7 and Q15 inputs:

$$c_i = a_i \times b_i$$

As introduced in Chapter 12.1.6, the product of multiplying two $Q_{m.n}$ numbers can be obtained by multiplying two integers corresponding to these two fixed-point numbers, and then shift the product right by n bits.

Therefore, when multiplying two Q7 numbers, the product of corresponding 8-bit signed integers should be arithmetically shifted right by 7 bits.

```

vector_mult_Q7 PROC
    EXPORT vector_mult_Q7

    ; r0 = pSrcA, pointer to source Q7 array A
    ; r1 = pSrcB, pointer to source Q7 array B
    ; r2 = pDst, pointer to destination Q7 array
    ; r3 = # of Q7 values in source array

    PUSH    {r4-r8, lr}
    MOV     r4, #0                ; Initialize loop counter

    B       check

loop    LDRSB r5, [r0], #1        ; 1st Q7 of A
        LDRSB r6, [r1], #1        ; 1st Q7 of B

        SMULBB r5, r5, r6          ; r5 = r5[B]*r6[B]

        ; Shift product right by 7 bits and saturate result to 8 bits
        SSAT   r5, #8, r5, ASR #7
        STRB   r5, [r2], #1        ; Store results to destination array

        ADDS   r4, r4, #1          ; Increase loop counter
check   CMP    r4, r3              ; Compare with array size
        BLT    loop

        POP    {r4-r8, PC}        ; Recover registers and return

    ENDP

```

Example 24-25. Multiplying the corresponding values in two Q7 arrays

Example 24-25 shows the assembly program of pairwise multiplication between two Q7 arrays. In each loop, the program loads one byte from each array, multiply them, and store the product byte back to the data memory.

This example cannot leverage SIMD instructions to perform multiple pairs of Q7 values in parallel. The reason is that all SIMD parallel multiplication instructions supported in Cortex-M4 and M7 also perform accumulations simultaneously. Nevertheless, no accumulation is required in this example.

Similarly, when two Q15 numbers are multiplied, the product of their corresponding sized 16-bit integers should be right shifted right by 15 bits.

In the following sample program, we multiply two pairs of Q15 numbers in each loop.

```

vector_mult_Q15 PROC
    EXPORT vector_mult_Q15

    ; r0 = pSrcA, pointer to source Q15 array A
    ; r1 = pSrcB, pointer to source Q15 array B
    ; r2 = pDst, pointer to destination Q15 array
    ; r3 = # of Q15 values in source array

    PUSH    {r4-r9, LR}          ; Preserve registers
    MOV     r4, #0                ; Initialize loop counter

    B       check

loop    LDR     r5, [r0], #4        ; Load two Q15 values from A
        LDR     r6, [r1], #4        ; Load two Q15 values from B

        SMULTT  r7, r5, r6          ; Multiply one pair
        SMULBB  r5, r5, r6          ; Multiply the second pair

        SSAT    r8, #16, r7, ASR #15 ; Saturate result to 16 bits
        SSAT    r7, #16, r5, ASR #15 ; Saturate result to 16 bits
        PKHBT   r9, r7, r8, LSL #16  ; Pack two Q15 result into a word
        STR     r9, [r2], #4        ; Store results

        ADDS    r4, r4, #2          ; Increase loop counter

check   CMP     r4, r3              ; Compare with array size
        BLT     loop

        POP     {r4-r9, PC}        ; Recover registers and return

    ENDP

```

Example 24-26. Multiplying corresponding values in two Q15 arrays

24.9.7 Vector Dot Product

The following defines the dot product of two vectors.

$$P = A \cdot B = \sum_{i=0}^{n-1} A_i B_i = A_0 B_0 + A_1 B_1 + \cdots + A_{n-1} B_{n-1}$$

To increase the precision, the product is in Q31 format even though inputs are in Q7. The product is Q63 if inputs are Q15.

The following program calculates the dot product of two Q7 arrays.

```

vector_dot_prod_Q7 PROC
    EXPORT vector_dot_prod_Q7

    ; r0 = pSrcA, pointer to source Q7 array A
    ; r1 = pSrcB, pointer to source Q7 array B
    ; r2 = # of Q7 values in source array

    PUSH    {r4-r9, LR}        ; Preserve registers
    MOV     r9, #0              ; Initialize accumulator
    MOV     r4, #0              ; Initialize loop counter

    B       check

loop LDR     r5, [r0], #4        ; r0 = pSrcA, Load four Q7 values from A

    SXTB16 r7, r5, ROR #8       ; Extract two Q7 values and make them Q15
    SXTB16 r8, r5               ; Extract the remaining two values

    LDR     r6, [r1], #4        ; r1 = pSrcB, Load four Q7 values from B

    SXTB16 r5, r6, ROR #8       ; Extract two Q7 values and make them Q15
    SXTB16 r6, r6               ; Extract the remaining two values

    SMLAD   r9, r7, r5, r9       ; r9 = r9 + r7[T]*r5[T] + r7[B]*r5[B]
    SMLAD   r9, r8, r6, r9       ; r9 = r9 + r8[T]*r6[T] + r8[B]*r6[B]

    ADDS    r4, r4, #4          ; Increase loop counter
check CMP   r4, r2              ; r2 is the array size
    BLT     loop

    MOV     r0, r9              ; Return result in r0
    POP     {r4-r9, PC}        ; Recover registers and return

    ENDP

```

Example 24-27. Dot product of two Q7 arrays

Computing the dot product of two Q15 arrays is given below.

```

vector_dot_prod_Q15 PROC
    EXPORT vector_dot_prod_Q15

    ; r0 = pSrcA, pointer to source Q15 array A
    ; r1 = pSrcB, pointer to source Q15 array B
    ; r2 = # of Q15 values in source array

    PUSH    {r4-r8, LR}      ; Preserve registers
    MOV     r7, #0            ; Lower word of accumulator
    MOV     r8, #0            ; Upper word of accumulator
    MOV     r4, #0            ; Initialize loop counter

    B       check

loop  LDR     r5, [r0], #4      ; r0 = pSrcA, load two Q15 values
      LDR     r6, [r1], #4      ; r1 = pSrcB, load two Q15 values

      SMLALD r7, r8, r5, r6    ; r8:r7 = r8:r7 + r5[T]*r6[T] + r5[B]*r6[B]

      ADDS    r4, r4, #2        ; Increase loop counter
check CMP     r4, r2            ; Compare with array size
      BLT     loop

      MOV     r0, r7            ; Return result in r1:r0
      MOV     r1, r8
      POP     {r4-r8, PC}      ; Recover registers and return

    ENDP

```

Example 24-28. Dot product of two Q15 arrays

24.9.8 Vector Min and Max

The following program finds both the global min and max of a Q7 array. The result is returned in register r0, with the max stored at the upper halfword and the min at the lower halfword.

```

vector_minmax_q15 PROC
    EXPORT vector_minmax_q15

    ; r0 = pSrc, pointer to source Q15 array
    ; r1 = # of Q15 values in source array

    PUSH    {r4-r6, lr}

    LDR     r2, [r0]           ; Initialize min
    LDR     r3, [r0]           ; Initialize max

```

```

MOV    r4, #0           ; Initialize loop counter
B      check

loop   LDR    r5, [r0], #4 ; Load two Q15 values, post-index
        SSUB16 r6, r3, r5 ; Compare current values with max
        SEL    r3, r3, r5 ; Select new max values
        SSUB16 r6, r5, r2 ; Compare current values with min
        SEL    r2, r2, r5 ; Select new min values
        ADDS   r4, r4, #2 ; Increase loop counter
check  CMP    r4, r1      ; Compare with array size
        BLT    loop

        ; Find the max of two halfwords of r3
        LSR    r6, r3, #16
        MOV    r5, #0xFFFF
        AND    r3, r3, r5
        SSUB16 r5, r3, r6
        SEL    r3, r3, r6

        ; Find the min of two halfwords of r2
        LSR    r6, r2, #16
        MOV    r5, #0xFFFF
        AND    r2, r2, r5
        SSUB16 r5, r6, r2
        SEL    r2, r2, r6

        ; Pack max to upper halfword and min to low halfword
        PKHBT  r0, r2, r3, LSL #16
        POP    {r4-r6, pc}
        ENDP

```

Example 24-29. Find the min and max in a Q15 array

24.10 Exercises

1. Find out the value of register r1.

	USAT16 r1, #1, r0	SSAT16 r1, #1, r0
r0 = 0x00000000	r1 =	r1 =
r0 = 0x000F000F	r1 =	r1 =
r0 = 0x00010001	r1 =	r1 =
r0 = 0x00020002	r1 =	r1 =
r0 = 0x00100010	r1 =	r1 =
r0 = 0xFFFFFFFF	r1 =	r1 =

2. Assume the value in register r1 and r0 is 0xF880FF7F. What is the value in register r2 after running each of the following instruction? What are the APSR.GE bit flags?

- (1) SADD8 r2, r1, r0
- (2) UADD8 r2, r1, r0
- (3) QADD8 r2, r1, r0
- (4) UQADD8 r2, r1, r0
- (5) SHADD8 r2, r1, r0
- (6) UHADD8 r2, r1, r0

3. Assume r1 = 0x7FFF1000 and r0 = 0xFFFF7FFF, what is the value in register r2 after running each of the following instructions?

- (1) SSUB16 r2, r1, r0
- (2) USUB16 r2, r1, r0
- (3) QSUB16 r2, r1, r0
- (4) UQSUB16 r2, r1, r0
- (5) UHSUB16 r2, r1, r0
- (6) SHSUB16 r2, r1, r0

4. Assume r1 = 0x0001FFFF and r0 = 0xFFFF0001, what is the value in register r2 after running each of the following instructions?

- (1) SMULBB r2, r1, r0
- (2) SMULBT r2, r1, r0
- (3) SMULTB r2, r1, r0
- (4) SMULTT r2, r1, r0
- (5) SMUAD r2, r1, r0
- (6) SMUADX r2, r1, r0
- (7) SMUSD r2, r1, r0
- (8) SMUSDX r2, r1, r0

5. Assume r1 = 0x22221111 and r0 = 0x44443333, what is the value in register r2 after running each of the following instructions?

- (1) PKHTB r2, r1, r0
- (2) PKHBT r2, r1, r0
- (3) PKHTB r2, r1, r0, ASR #16
- (4) PKHBT r2, r1, r0, LSL #16

6. Write an assembly program that uses SEL and packing instructions to achieve the following: pack the max of the upper halfwords of registers r1 and r0 and the min of the lower halfwords of registers r1 and r0 into register r3.

- r3[31:16] = max(r1[31:16], r0[31:16])
- r3[15:0] = min(r1[15:0], r0[15:0])

7. Convert a color image to a grayscale image. Suppose each pixel in the image has three 8-bit byte values: red (R), green (G), and blue (B). The image has 1024 pixels. All red values are stored in an array starting at the memory address 0x20000000, all green values starting at 0x20004000, and all blue values starting at 0x20008000. Write an assembly program that calculates the grayscale value of each pixel by using the following formula.

$$\text{Greyscale} = (R \times 77 + G \times 151 + B \times 28) / 256$$

8. Loop unrolling is a commonly used compiler technique to speed up the application performance. The basic idea is to duplicate the original loop body multiple times and reduce the number of loops. The following example illustrates the key idea of loop unrolling. The unrolled loop runs faster because fewer branch instructions are executed, and a larger loop body gives compilers more opportunity to optimize the performance.

Normal loop	Loop unrolling by four times
<pre>int sum = 0; for (i = 0; i < 100; i++) sum += a[i];</pre>	<pre>int sum = 0; for (i = 0; i < 25; i++) { sum += a[4*i]; sum += a[4*i + 1]; sum += a[4*i + 2]; sum += a[4*i + 3]; }</pre>

Rewrite subroutine `vector_dot_prod_Q15` given in Example 24-28 and unroll the loop twice. Assume the number of Q16 numbers in each array is a multiple of four.

9. Unroll the loop of `vector_mean_Q15` given in Example 24-24 four times. Assume the number of Q16 values in the input array is a multiple of eight.
10. Implement the following assembly subroutine that multiplies each element of a Q15 array with a Q15 constant. The results should be in the Q15 format.

```
vector_Q15_scale PROC
    ; r0 = memory address of Q15 array
    ; r1 = array size
    ; r2 = Q15 constant

    ENDP
```